ALIENs for Continuous Time Economies.*

Goutham Gopalakrishna[†], Yuntao Wu[‡]

April 13, 2026

First version: May 21, 2021

Abstract

This paper builds ALIENs (Active Learning Inspired Equilibrium Nets), that extend deep-learning methods for solving continuous-time equilibrium models with high-dimensional states, aggregate shocks, and nonlinear dynamics. The approach combines time-stepping with active learning to turn nonlinear problems into sequences of contraction mappings, improving stability and convergence. By focusing computation on economically important regions of the state space, ALIENs increase accuracy and efficiency. The method is validated across applications ranging from heterogeneous-agent models with free boundaries to high-dimensional asset-pricing models. Additionally, the paper introduces Deep-Macrofin+, a numerical library that facilitates the implementation of these techniques for researchers.

Keywords: Deep learning, global dynamics, computational methods.

*We are very grateful to the comments and discussion from Markus Brunnermeier, Pierre Collin-Dufresne, Marlon Azinovic, Jonathan Payne, Moritz Lenel, Sebastian Merkel, Galo Nuno, Simon Scheidegger, and Yucheng Yang, and participants at the University of Zurich workshop, Princeton BCF workshop, and EPFL brownbar series. We also thank FinHub at Rotman School of Management for the computational resources.

[†]University of Toronto, Rotman School of Management.
Email: goutham.gopalakrishna@rotman.utoronto.ca

[‡]University of Toronto, Electrical and Computer Engineering.
Email: winstonyt.wu@mail.utoronto.ca

1 Introduction

The past decade has seen a surge in macroeconomic and asset pricing models to capture nonlinear global dynamics. Models in continuous time offer the ability to capture complex interactions due to their tractability and offer portfolio choices in closed form compared to the discrete time counterparts. While continuous-time models characterizing the global dynamics are certainly an improvement over models with linearized solutions, most of the papers resort to low-dimensional state spaces due to computational bottlenecks. The difficulty is amplified when the state variables are endogenous and correlated, and the equilibrium quantities exhibit stark non-linearities. In problems with aggregate shocks and long-lived assets, the standard finite difference method breaks down even with two state variables, since it is difficult to preserve the monotonicity of finite difference schemes.[1] Therefore, it is not just the curse of dimensionality, but the combination of nonlinear dynamics and high dimensions that is problematic in these models. Recent advances have used neural networks that alleviate some of these problems.[2] Neural network methods work in theory but are often hard to implement in practice. The flexibility that the neural network methods provide is both a blessing and a curse since there are many hyperparameter choices to make that affect the neural network approximations to equilibrium quantities.

This paper adds to the existing neural networks based numerical literature by proposing a deep learning method with high stability and convergence properties through active learning. The methodology proposed is tailored to solve continuous-time equilibrium economic models

[1] Preserving monotonicity is important when there are cross partial derivatives in the HJB equation. See, for example, D'avernas, Petersen and Vandeweyer (2023) and Merkel (2020) for details.

[2] See, for example, Duarte, Duarte and Silva (2024), Maliar, Maliar and Winant (2021), Azinovic, Gaegauf and Scheidegger (2022), Han, Yang and E (2021), Gu, Laurière, Merkel and Payne (2024), Huang (2023), among others.

that feature aggregate shocks, long-lived assets, and high nonlinearities in policy and value functions. The contribution of this paper is twofold. First, we show through examples how a time-stepping based method enhances the stability of continuous-time economic models when the value and policy functions are approximated using deep neural networks with multilayered perceptrons (MLPs). The enhanced properties arise because the time-stepping procedure transforms the numerical problem to resemble a contraction mapping. We formalize this using the contraction mapping principle and apply the technique to a class of low- and high-dimensional problems where policy functions are highly nonlinear. The results show that our solution technique performs better than both the traditional finite difference method and the basic neural network method. In particular, active learning plays a critical role in achieving convergence by strategically selecting training points that guide the algorithm to learn equilibrium relationships within the most economically relevant areas of the state space. Second, the paper offers a numerical library, *Deep-Macrofin+*, powered with active learning. Compared to packages with traditional numerical methods like finite difference, it has the potential to solve higher dimensional models, with less restrictions in the types of derivatives in HJB equations. Compared with existing neural network based packages, it is more targeted at economic models and integrates various active learning techniques for better convergence. Lastly, it offers a more user-friendly approach with latex and string equation inputs.[3] To the best of our knowledge, this is the first paper to use neural networks with *active learning* in continuous-time models as well as offer a user-friendly toolbox tailored for high dimensional equilibrium economic models.

While our method follows the deep learning-based solution techniques in the literature in

[3]Link to software, documentation and replication package can be found in Appendix F.

employing neural networks that are trained to solve a supervised learning problem, there are two main differences. First, we implement a time-stepping scheme with neural networks that ensures stability, especially in problems where policy functions are highly nonlinear. Second, the training points are sampled from the most important part of the subspace depending on the economic and computational complexity of the problem. We focus on two types of active learning mechanisms (i) "residual-based" and (ii) "loss weight-based".

In the "residual-based" learning, training points are drawn from regions where the out-of-sample losses are large. That is, the subspace where equilibrium conditions are violated is given more importance. In the "loss weight-based" learning, each loss function is assigned dynamic weights over training iterations. The weights are assigned on the basis of the relative progression in the loss. While the residual-loss method focuses on the subspace with larger residual, the loss weight-based method focuses on the *change* in the loss over time. That is, individual losses in a multi-loss problem that decrease slower get a higher weight. The intuition is that the equilibrium relationships that are harder to learn relative to other relationships get a higher relative weight dynamically over the training iterations. Ultimately, both methods focus on improving convergence dynamically without the need for manual intervention in the intermittent training procedure.

We demonstrate our technique using a few applications with varying degrees of complexity. In the first application, we solve an infinite-horizon heterogeneous agent macro-finance model with short-selling constraints that feature an endogenous free-boundary, rendering the policy functions non-smooth.[4] In theory, MLPs can be made infinitely deep or wide

[4]Approximations using deep neural networks often fail because the universal approximation theorem for MLPs with smooth activation functions assumes that the approximated functions and their partial derivatives are continuous. (Hornik, Stinchcombe and White (1989); Hornik (1991)). In contrast, the approximation theorem for non-smooth activation functions, such as ReLU, assumes that the approximated function is

to minimize approximation errors. However, accurately approximating a function around a jump discontinuity is challenging, especially in equilibrium models that involve market clearing on long-lived assets (see Gopalakrishna, Gu and Payne (2026), for example). Our time-stepping method introduces an artificial time dimension and reformulates the stationary equilibrium problem as the steady state of an evolution equation. The resulting artificial-transient formulation allows the solution to be computed sequentially, solving a sequence of intermediate problems over a finite horizon. While the HJB equation remains nonlinear, the time-stepping procedure yields incremental updates that improve stability and convergence of the neural network training. Under suitable conditions on the HJB operator, the approximation converges to the stationary equilibrium with sufficient accuracy. The theoretical basis for the convergence comes from the fact that the time-stepping scheme converts the problem into a sequence of contraction mappings.

In the second application, we solve an optimal stopping time problem with a smooth transition at the free boundary, and extend to higher dimensions. We demonstrate the importance of learning specifically around the free boundary to smooth the loss progression, especially on derivatives.

In the third application, we solve the model without short-selling constraint but with contractible idiosyncratic and stochastic volatility. The model features high non-linearities due to stochastic volatility and concentrated long-lived asset positions in one group of agents. This is a multi-loss problem with first-order conditions with respect to consumption and portfolio choices, goods market clearing, and capital market clearing conditions entering the loss function. We show that a loss-weight–based active learning scheme improves convergence

piecewise linear Petersen and Zech (2024).

by directing learning toward conditions that exhibit slower progress.

Lastly, we test our method in a high-dimensional model where we introduce asset heterogeneity a la Martin (2013). We show that our method solves up to 50 trees with an accuracy (HJB error) of order 10^{-8}, where active learning leads to a better accuracy of policy functions. To understand why active learning helps in this high-dimensional model, we reduce the dimensionality of the same model and study a 3-tree version. Analyzing this model reveals that the residual-based active learning samples training points that are concentrated at the boundaries of the state space, where the equilibrium relationships are the most important and yet get violated when approximated with a basic neural network without active learning.

We build an active learning based numerical library called "Deep-Macrofin+" that facilitates researchers to solve such models with nonlinear policy functions without having to write deep learning training modules. The HJB equations and algebraic equations are treated as loss functions that the training algorithm optimizes to approximate the value and policy functions, following the core idea of deep learning-based solution methods in the literature. In addition to this standard feature, the tool incorporates two types of active learning methods, a loss-weight-based method and a residual-based method. We demonstrate its usage in solving several macro-finance models in the GitHub repository, in addition to the ones in this paper.

Literature review: There have been many papers with deep learning-based techniques to solve discrete-time economic problems (Maliar and Maliar (2015), Azinovic et al. (2022), Maliar et al. (2021), Han et al. (2021), Bretscher, Fernández-Villaverde and Scheidegger

(2022) etc.) Recently, few papers have emerged that solve continuous-time models by training neural networks on simulated points of discrete time approximations (Han, Jentzen and E (2018), Huang (2023), etc.). These techniques share the same spirit as the discrete-time methods in the sense that equilibrium functions are approximated on simulated paths after discretizing them. This paper fits within a growing yet nascent literature that solves continuous-time models by taking an analytic PDE approach. The papers closest in spirit are Duarte et al. (2024), Gopalakrishna et al. (2026), Gu et al. (2024), Sauzet (2021), and Payne, Rebei and Yang (2024). Duarte et al. (2024) takes a reinforcement learning-based approach by converting the equilibrium economic model into an optimal control problem and then solves it as a supervised learning problem. Following Han et al. (2021), we use HJB equation error on out-of-sample datapoints to evaluate the performance. Our paper differs by introducing active learning, allowing for flexibility to tackle discontinuous policy functions and multi-loss objective functions leading to improved convergence properties. Since the time this paper has been made publicly available, the idea that smart sampling from the state space is required to improve convergence properties has proven to be useful for macro models. For example, Gu et al. (2024) finds that a sampling method similar to the residual-based active sampling in this paper leads to better convergence in a continuous-time version of the Krusell-Smith model. In Fernández-Villaverde, Hurtado and Nuño (2023), the law of motion of aggregate wealth is solved using neural network approximations, while value functions are solved using traditional finite difference schemes. Moreover, their model does not feature "long-lived" capital, which has proven to be tricky to handle in high dimensions with aggregate shocks (see Gopalakrishna et al. (2026), for example). The loss-weight based active learning relates to Bretscher et al. (2022) where the weights in the loss function are

normalized. The difference is that in this paper, the loss weights are computed based on the relative loss progression over epochs with a memory decay.

There is a substantial literature in computational physics and applied mathematics to approximate PDEs and HJBs using neural networks, starting from Sirignano and Spiliopoulos (2018) and Raissi, Perdikaris and Karniadakis (2019). Sirignano and Spiliopoulos (2018) proposes a deep Galerkin method to solve PDEs in high dimensions and incorporates Monte Carlo methods to compute second-order derivatives to speed up computation. In contrast to these papers, the framework that we propose is suited to solving problems in financial economics. For instance, many problems in macro-finance and asset pricing come with endogenous state variables that are often correlated - a feature that the models in applied mathematics do not typically deal with. Moreover, the non-linearity of the PDEs in the economic models comes from the fact that the advection, diffusion, linear, and cross-term coefficients of the PDE are endogenously dependent on the equilibrium policy functions. Lastly, the usage of sample points using active learning in this paper relates to the literature on adaptive sparse grids. For example, Brumm and Scheidegger (2017) and Schaab and Zhang (2022) use adaptive sparse grids to solve dynamic economic models, whereas Bungartz, Heinecke, Pflüger and Schraufstetter (2012) solves option pricing models using the finite element method.

While there have been many attempts at solving economic models using neural networks, the literature on providing a user-friendly toolbox to solve economic models has been sparse. Lu, Meng, Mao and Karniadakis (2021) offers a physics-informed neural network (PINN) library to solve both forward and inverse PDE problems using deep neural networks, but no active learning is implemented. D'avernas et al. (2023) provides a dedicated library to

solve macro-finance equilibrium problems in continuous time with one or two state variables, but does not expand to higher dimensions or problems with more complicated equilibrium relations. Our toolbox fills this gap.

2 Methodology

In this section, we present the deep learning methodology. We start by briefly describing neural network approximations, and then formalize the time-stepping scheme. We then explain active learning implementation and present algorithms.

2.1 Neural network approximations

Let $J(\boldsymbol{x})$ denote the function that takes as input $\boldsymbol{x}$ and needs approximation. Assume that $J : \mathbb{R}^d \to \mathbb{R}$ is continuous, which is required to apply the universal approximation theorem of MLPs with continuous activation functions (such as *tanh*, *SiLu*, and *SoftPlus*). Moreover, if we aim to approximate derivatives of J, then J must be continuous up to the order of derivatives we want to approximate (*e.g.* if we want to approximate J upto second-order derivatives, then J must be C^2). When these regularity conditions are violated, MLPs tend to produce poor approximations. Let $\hat{J}(\boldsymbol{x})$ denote the neural network approximation represented as a parametric function built using a sequence of linear and non-linear transformations. That is, we have

$$\hat{J}(\boldsymbol{x}) = W^L(\sigma(W^{L-1}(\sigma(...(\sigma(W^0\boldsymbol{x} + b^0)) + b^{L-1})...))) + b^L,$$

where σ is the activation function, W^l is the weight matrix for the l-th layer, b^l is the bias vector for the l-th layer, and L is the number of layers. This is a feedforward neural network with d inputs, L hidden layers, and a single output. There are other complex architectures that can be used, such as DGM (Sirignano and Spiliopoulos (2018)) and ResNet. While these alternative architectures can be helpful for certain economic models in lower dimensions, they often become computationally expensive when extended to higher-dimensional settings, particularly because approximating second-order derivatives significantly increases the computational burden. Therefore, we focus on the feedforward neural network for simplicity. The goal is to approximate unknown equilibrium functions in the economic model using neural networks with the above architecture, and train them by minimizing the residuals of the equilibrium conditions. The loss function is model-specific and may contain multiple terms, such as the residual of the HJB equation, the residual of the boundary conditions, the residual of the market clearing conditions, etc. Training points ($\boldsymbol{x}_i$) are sampled from the state space, and the network parameters are optimized to minimize the residuals evaluated at these points. This follows the growing literature on using neural networks to solve equilibrium problems in economics. Our innovation is to incorporate time-stepping and active learning to improve the convergence properties of the training algorithm.

2.2 Solution method

Let θ be the parameters of the neural network (weights of the matrices W and biases b), and $\mathcal{T}$ be the training data. The overall loss function is defined as:

$$\mathcal{L}(\theta, \mathcal{T}) = \sum_i \lambda_i \mathcal{L}_i(\theta, \mathcal{T}),$$

where $\mathcal{L}_i(\theta, \mathcal{T})$ are the mean squared residuals for boundary/initial conditions, HJB equations, inequality constraints, and constraint-activated systems, and λ_is are corresponding weights. Under continuity assumptions, the neural network approximation converges to the equilibrium functions.

Neural network methods that are formulated to minimize $\mathcal{L}(\theta, \mathcal{T})$ rely on information derived from the model's first-order conditions and the associated HJB equations. Rather than enforcing the governing equations pointwise, these methods seek a parametric function Φ_θ that minimizes residuals over a set of training points $\mathcal{T}$. Consequently, the resulting approximation is generally not expected to satisfy the model's conditions exactly at every point in the state space, but instead in an average or residual-minimizing sense. The loss function $\mathcal{L}(\theta, \mathcal{T})$ quantifies how well Φ_θ satisfies the algebraic constraints, boundary and initial conditions and HJB equations. Importantly, this procedure does not assume prior knowledge of the true solution f; the approximation is obtained solely by enforcing consistency with the model-implied conditions. Assuming the following conditions hold: 1) the universal approximation theorem for neural networks; 2) smoothness of the neural network approximation with non-$ReLU$ activations; 3) continuity properties of the PDEs; and 4) the existence of a unique solution to the PDE system, we can apply the approach

outlined in Sirignano and Spiliopoulos (2018) to demonstrate that $\Phi_\theta \to f$. In practice, rather than a wide neural network with many neurons in a single layer, a deep neural network with multiple layers tends to achieve better convergence. The true solution f, or its derivatives, need not be continuous or well-defined across the entire domain. For existence of a solution, it is sufficient that these derivatives be defined and continuous almost everywhere. However, when neural networks with smooth activation functions are used for approximation, capturing discontinuities or kinks becomes challenging. We formalize this in Proposition 1. This situation is analogous to the Fourier approximation of functions with jump discontinuities, where the Gibbs phenomenon produces persistent oscillations near the discontinuity. Similarly, because practical neural networks are finite in width and depth, the approximation $\Phi(x)$ deviates from $f(x)$ in a neighborhood of the discontinuity. In effect, the network smooths out sharp features, much like a low-order truncated Fourier series struggles to accurately represent functions near jump points.

Proposition 1. *Let $f : [0,1] \to \mathbb{R}$ have a jump discontinuity at $x_0 \in (0,1)$, with finite one-sided limits*

$$f(x_0^-) := \lim_{x \uparrow x_0} f(x), \qquad f(x_0^+) := \lim_{x \downarrow x_0} f(x).$$

Let $\Phi_\theta : [0,1] \to \mathbb{R}$ be a continuous neural network trained by minimizing a squared-loss objective

$$\mathcal{L}(\theta) = \int_0^1 \left|\Phi_\theta(x) - f(x)\right|^2 d\mu(x),$$

where μ is a probability measure whose density is continuous and strictly positive in a neighborhood of x_0. Suppose the family $\{\Phi_\theta\}$ is uniformly equicontinuous on $[0,1]$ (through

a uniform Lipschitz bound on the networks, which arises naturally from bounded network parameters). Then any sequence of minimizers $\{\Phi_{\theta_n}\}$ *converging uniformly on compact subsets of* $[0,1] \setminus \{x_0\}$ *satisfies*

$$\lim_{n\to\infty} \Phi_{\theta_n}(x_0) = \frac{f(x_0^-) + f(x_0^+)}{2}.$$

Proof. See Appendix E. □

Algorithm 1 Time Stepping Scheme

Input: X: state variables with time t,
$J_i : X \to \mathbb{R}$: agent value variables,
$E_j : X \to \mathbb{R}$: endogenous variables,
Output: Trained approximations $\hat{J}_i$, $\hat{E}_j$.

$\tau \leftarrow 0, \hat{J}_{i,\tau=0} = 1, \hat{E}_{i,\tau=0} = 1$ {Initialize as constant}
while True **do**
  Sample $X = (x_0, ..., x_n, t)$ from domain ($x_0, ..., x_n$ are defined by the problem domain, $t \in [0,1]$)
  Embed boundary conditions: $J_{i,\tau+1}(t=1) = \hat{J}_{i,\tau}$, $E_{i,\tau+1}(t=1) = \hat{E}_{i,\tau}$ {At maximum time, the functions should satisfy the value from the previous step}
  while True **do**
    Update variables using neural networks, with $\frac{\partial J_i}{\partial t}$ and $\frac{\partial E_i}{\partial t}$ integrated.
    Compute loss on boundary conditions, endogenous equations, HJB equations and systems
    Compute total loss
    if inner_iter $\geq$ max_inner_loop OR inner loss converges **then**
      break
    end if
  end while
  $\hat{J}_{i,\tau+1} \leftarrow J_{i,\tau+1}(t=0)$, $\hat{E}_{i,\tau+1} \leftarrow E_{i,\tau+1}(t=0), \tau \leftarrow \tau + 1$
  if outer_iter $\geq$ max_outer_loop OR $\hat{J}_i$, $\hat{E}_i$ converge **then**
    break
  end if
end while

Our general approach to solving equilibrium models is to approximate stationary infinite-horizon equilibria by introducing an artificial finite time horizon T. That is, we augment the

state space with a false transient time variable $t \in [0, T]$. The objective is to evolve the system in this artificial time dimension until it converges to a stationary solution. In continuous time models, this amounts to modifying the HJB equations by adding a time derivative when computing the drifts using Ito's lemma. The terminal condition at $t = T$ is initially fixed (typically to a constant), and Φ is solved backward in time. The terminal value itself is not economically meaningful; what matters is that the system converges to a stationary point at which the artificial time derivative $\partial_t \Phi$ vanishes. Since the equilibrium constraints must hold for all t, the only structural modification relative to the stationary formulation is the inclusion of the time derivative in the HJB equations. Algorithm 1 summarizes this time-stepping procedure implemented with neural networks. The algorithm consists of two nested loops. In the inner loop, the neural networks are trained to minimize the loss function $\mathcal{L}(\theta, \mathcal{T})$, enforcing the equilibrium conditions over the augmented domain $X = (x_0, ..., x_n, t)$. Given a fixed terminal boundary condition at $t = T$, this produces an approximation Φ_τ that satisfies an evolution equation of the form $\partial_t \Phi = \mathcal{F}(x, t, \Phi)$. In the outer loop, the terminal boundary condition is updated according to the matching rule $\Phi_{\tau+1}(x, T) = \Phi_\tau(x, 0)$, propagating information across iterations. Conceptually, this procedure mirrors Value Function Iteration (VFI). The outer loop in our algorithm iteratively updates the terminal condition so that the time evolution converges to a fixed point that is invariant in the artificial time dimension. Proposition 2 formalizes this idea. The key object is the *terminal-to-initial solution map* S, which takes a terminal boundary condition g and returns the solution evaluated at $t = 0$ after solving the evolution equation. The outer loop of Algorithm 1 iterates $g_{\tau+1} = S(g_\tau)$. Under contraction of S, which in economic models is naturally provided by discounting factor e^{-rT}, the Banach fixed-point theorem guarantees that the sequence $\{g_\tau\}$ converges to

a unique fixed point g^*, independent of t. The corresponding solution satisfies $\partial_t \Phi^* = 0$, and Φ^* solves the original stationary equilibrium problem. Thus, the time-stepping scheme can be interpreted as a neural-network implementation of a contraction-based fixed-point iteration in function space, where stationarity emerges endogenously as the limit of the artificial transient dynamics.

Proposition 2. *Let Ω be the state space, $T > 0$ the artificial time horizon, and $\mathcal{F}$ a differential operator. Consider the evolution equation*

$$\partial_t \Phi(x,t) = \mathcal{F}[x,t,\Phi(\cdot,t)], \qquad (x,t) \in \Omega \times [0,T],$$

and define the terminal-to-initial map $S : C(\Omega) \to C(\Omega)$ by $S(g)(x) := \Phi_g(x,0)$, where Φ_g denotes the unique solution satisfying the terminal condition $\Phi_g(\cdot,T) = g$.

Assume that (1) For each terminal condition $g \in C(\Omega)$, the evolution equation admits a unique solution Φ_g on $\Omega \times [0,T]$; (2) At each outer iteration τ of Algorithm 1, the inner-loop training produces an approximation Φ_τ satisfying

$$\lim_{n\to\infty} \sup_{(x,t)\in\Omega\times[0,T]} |\partial_t \Phi_\tau(x,t) - \mathcal{F}[x,t,\Phi_\tau(\cdot,t)]| = 0.$$

Then: 1. The outer-loop terminal condition sequence $\{g_\tau\}$ defined by $g_{\tau+1} = S(g_\tau)$ converges uniformly to a unique fixed point $g^* \in C(\Omega)$. 2. The corresponding solution Φ^* with $\Phi^*(\cdot,T) = g^*$ satisfies $\Phi^*(x,0) = \Phi^*(x,T)$ for all x. 3. If, in addition, the stationary equation $\mathcal{F}[x,\Phi] = 0$ admits a unique solution, then Φ^* is independent of t, $\partial_t \Phi^* \equiv 0$, and Φ^* solves the original stationary problem.

Proof. See Appendix E. □

2.3 Active learning

In principle, neural networks can be trained using points drawn uniformly from the state space in our time-stepping method. There are two potential bottlenecks with this kind of sampling. First, when policy functions are discontinuous and/or highly nonlinear, neural networks struggle to approximate them accurately. Second, when the number of state variables is large, it is computationally infeasible to sample points from the entire state space. Researchers in the past have tackled this problem by sampling from ergodic distribution of the state variables instead (Azinovic et al. (2022), Duarte et al. (2024), etc.) or sampling from the Kolmogorov forward equation (Gu et al. (2024)). Our active learning complements both these approaches. In the first case, active learning improves the accuracy of the neural network approximation by focusing more on points that have violated equilibrium conditions as a result of discontinuity and/or nonlinearity of policy functions. In the second case, active learning helps reduce the computational burden by playing the same role *among* the training points drawn from the ergodic distribution. Thus, active learning is agnostic about the type of sampling but acts as a guiding mechanism for choosing the right training points. We implement two active learning methods (i) residual-based, and (ii) loss weight-based.

Residual-based: In uniform sampling, all positions in the domain have an equal likelihood of being sampled. However, equilibrium models may be more challenging to learn in certain regions, such as those with higher curvature or where regime shifts and discontinuities occur, as in the free boundary models. Residual-based learning aims to improve the distribution of

training points by focusing on areas where the residual of the system is large. These are the areas where equilibrium conditions are violated. After a fixed number of learning iterations, a dense set of points is sampled from the state space, and their losses are computed. Points with higher residuals are then added to the training set, allowing the model to focus on the regions where equilibrium conditions are violated. Algorithm 2 shows the process.

Algorithm 2 Residual-based Learning

This only modifies the inner loop of Algorithm 1 (Line 5 to Line 12)

Additional Input: N_{sample}: number of times for residual-based sampling

while True **do**
  Update variables using neural networks, with $\frac{\partial J_i}{\partial t}$, and $\frac{\partial E_i}{\partial t}$ integrated.
  Compute losses derived from HJB equations and FOC
  Compute total loss
  if inner_iter % (max_inner_loop // N_{sample}) == 0 **then**
    Randomly sample a dense set of points in the state space for loss computation
    Retrieve k = batch_size/N_{sample} points with highest loss, add to the training set for further inner loop iterations.
  end if
  if inner_iter $\geq$ max_inner_loop OR inner loss converges **then**
    break
  end if
end while

Loss weight-based: For equilibrium models involving multiple components in the loss function, some components may be more important than others. The loss weight-based method assigns weights to each loss item based on how well it progresses. While residual-based active learning focuses on state space where equilibrium conditions have been violated in the current epoch, the loss weight-based method focuses on loss functions where the *change* in the loss has been slow. Intuitively, this forces the neural network method to focus on equilibrium relationships that are harder to learn. The implementation is inspired by

Bischof and Kraus (2025), where the weights are updated using the following equations:

$$\lambda_i^{bal}(t,t') = m\frac{\exp\left(\frac{\mathcal{L}_i(t)}{\beta\mathcal{L}_i(t')}\right)}{\sum_{j=1}^{m}\exp\left(\frac{\mathcal{L}_j(t)}{\beta\mathcal{L}_j(t')}\right)}$$

$$\lambda_i^{hist}(t) = \rho\lambda_i(t-1) + (1-\rho)\lambda_i^{bal}(t,0)$$

$$\lambda_i(t) = \alpha\lambda_i^{hist} + (1-\alpha)\lambda_i^{bal}(t,t-1)$$

m is the number of loss functions. $i \in \{1,...,m\}$ are indices for loss functions. β is softmax temperature. ρ is a Bernoulli random variable with $\mathbb{E}(\rho) \approx 1$. α is the exponential decay rate. The weights $\lambda_i^{bal}(t,t')$ depends on the ratio $\left(\frac{\mathcal{L}_i(t)}{\beta\mathcal{L}_i(t')}\right)$ and represents the relative improvement of loss functions between epochs t' and t. The parameter ρ randomly decides whether to carry forward the loss weight $\lambda_i(t-1)$ from the previous epoch $t-1$, or to recompute the loss weights based on the relative improvement from the beginning, denoted as $\lambda_i^{bal}(t,0)$. The parameter α regulates the exponential decay of the weight, balancing between the previous epoch's weights and those calculated for the current epoch. In our experiments, the parameter choices $\mathbb{E}(\rho) = 0.9999$, $\alpha = 0.999$, and $\beta = 0.1$ work well. These values produce a smooth evolution of the loss weights: the large α stabilizes the weights by smoothing them over time, while the high probability of ρ carrying forward the previous weights prevents frequent resets to the weights $\lambda_i^{bal}(t,0)$ based on the entire history. As a result, the weighting scheme primarily reacts to the relative improvement between consecutive epochs, while maintaining gradual and stable adjustments.

3 Applications

In this section, we apply our technique to solve four classes of equilibrium models that feature highly nonlinear policy functions. In each class of models, we demonstrate how our method leads to better convergence than a basic neural network method, and compare our solution to a traditional finite-difference method, whenever possible. All the losses are computed based on out-of-sample datapoints that are not part of the training set. In each experiment, every method is trained until convergence, defined as the point at which the loss no longer exhibits meaningful decay. The basic neural network baseline typically converges well within 20,000 epochs; additional training beyond this point does not reduce the loss further.

3.1 Free boundary model

Assume that time is continuous and the horizon is infinite (i.e. $t \in [0, \infty)$). There are two types of agents: households (h) and experts (e) who trade capital denoted by k_t that gets a stochastic depreciation shock driven by a standard Brownian motion Z_t. The output follows AK technology for simplicity. Agents have recursive preferences on consumption and choose optimal consumption, investment, and capital holdings by maximizing their lifetime utility subject to wealth constraints. Experts have a higher productivity rate of managing capital but cannot issue outside equity to households, reflecting the classic skin-in-the-game constraint found in macro-finance literature. In addition, agents cannot short-sell capital. These two assumptions mean that the market is incomplete, and there is an endogenous free-boundary in the state space at which the short-selling constraint binds. In the region where the short-selling constraint binds, the policy functions are linear. When

the constraint does not bind, agents trade capital with each other and policy functions are highly nonlinear. These features are present in many heterogeneous agent macro-finance models with incomplete markets and are often notoriously difficult to solve using traditional methods. We first solve a simpler one-dimensional model since it allows us to compare against the finite difference solution. We then solve a variant of this model with more shocks with two state variables to demonstrate the performance of our model. We keep the number of state variables low in this section to demonstrate our technique, and then proceed to higher-dimensional models in Section 3.4. Note that even though the number of state variables is low, the difficulty in solving it numerically comes from the fact that the model has kinks in multiple policy functions, with an endogenous barrier to be pinned down in equilibrium. D'avernas et al. (2023) shows that a naive finite difference method does not work for this model even in two dimensions with correlated shocks, reflecting the difficulty in solving models with long-lived assets and aggregate shocks.

The benchmark model follows Brunnermeier and Sannikov (2014) and a detailed description is found in Appendix A.1. The goal is to demonstrate the difficulty in approximating policy functions with a kink in a model with an endogenous free boundary. In this simple model, one remedy is to approximate functions with different neural networks in different regions in the state space. To reiterate, the purpose of using a simple model is to demonstrate that our method provides a solution that is close to the finite-difference approximation. The method is scalable to high dimensions, which will be demonstrated in Section 3.4 where we solve up to 50 dimensions. The model and calibration details are relegated to the Appendix A. The two approaches we consider are as follows.

1. Basic Neural Network method: train one neural network for each value and policy function, trying to capture the discontinuity.

2. Our Method: Train multiple neural networks for each value and policy function, one for each subdomain in the state space. The model is solved globally, and the free boundary is learned as part of the equilibrium solution. Concretely, we first train the neural networks in the region where $\psi < 1$. If the model in the region overshoots the boundary condition (*i.e.*, the implied ψ exceeds 1), we switch to the neural network corresponding to the shorting-constraint region. The boundary location thus emerges endogenously from the equilibrium conditions rather than being imposed exogenously. In the solution of the higher-dimension model, we apply the same model splitting strategy and demonstrate the performance improvement by time-stepping scheme with *residual-based* active learning following Algorithm 1 and 2.

3.1.1 Results

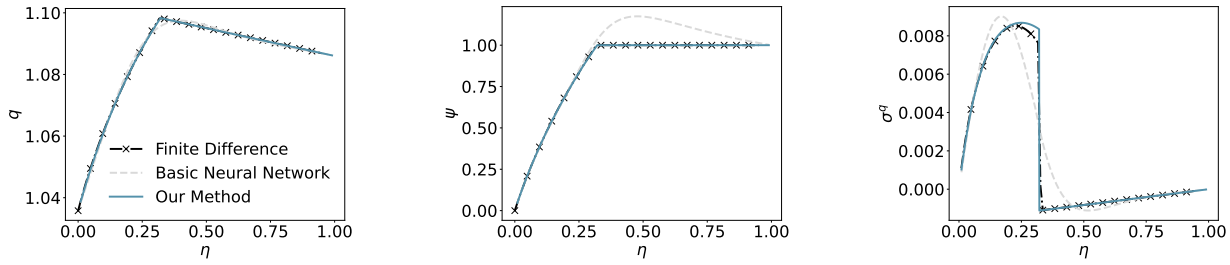
All neural network models use the $SiLU$ activation function. We use the Adam optimizer and train until convergence. Although the short-selling constraint region admits an analytical solution, we nevertheless learn the corresponding functions using neural networks to maintain consistency with the high dimensional model.

Figure 1 shows the equilibrium plots for q, ψ, σ^q using MLP models, together with finite-difference methods. Compared with the basic neural network method, our method provides more accurate policy functions. The loss from our method and the basic neural network method are comparable. However, the basic neural network method has low accuracy,

particularly around the free boundary. Appendix Figure A.1 shows that replacing *SiLU* with *ReLU* leads to inferior performance compared with the splitting method. For a fixed number of trainable parameters, the non-smooth nature of *ReLU* degrades the smoothness of the approximated functions.

Figure 1: Benchmark free boundary model equilibrium plots (SiLU activation). The MSEs for our method are 2.19×10^{-8} for price, 1.60×10^{-6} for experts capital shares, and 3.81×10^{-8} for price return diffusion.

(a) Price **(b)** Capital share: experts **(c)** Price return diffusion



We next proceed to solving a slightly richer model with two state variables and an endogenous barrier. This model is an extension of the simple benchmark model with an additional exogenous state variable, namely the productivity of experts $a_{e,t}$. We relegate the model details to the appendix and use this model as an apparatus to introduce our time-stepping method with active learning. The value functions are $J_{e,t}$ and $J_{h,t}$, and the policy functions are (q_t, χ_t, ψ_t). We parameterize each of these as a deep neural network object with MLPs. Each network contains 4 hidden layers with 64 neurons, activated by *SiLU*. At the end of the model, the outputs are restricted to be positive with *SoftPlus*. The learning rates are set to 0.001 for all models. 500 points are randomly sampled in each training time step. For the basic neural network method, we train for up to 20,000 epochs in each region; the total loss typically converges after 15,000 epochs, after which additional training yields no further improvement. For the time-stepping scheme, we train for 5,000 epochs in

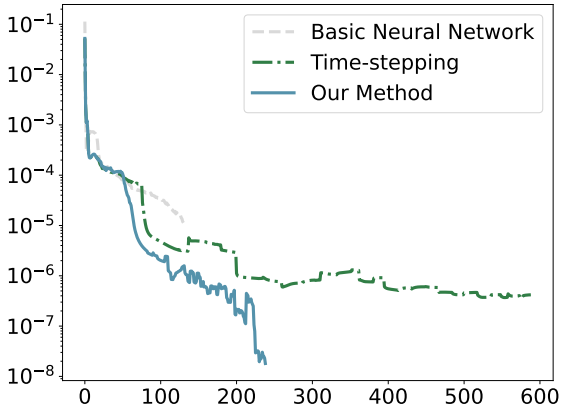
each time step and train for 70 time iterations. The 70 time iterations are unnecessary for shorting-constraint and no-constraint regions, where the equilibrium conditions are simpler, but we use the same configuration for consistency. We report the evolution of expert's HJB error evaluated at the iterations corresponding to the running minimum of the total loss, rather than along the full raw loss trajectory. This provides a cleaner visualization of convergence behavior by filtering out short-term oscillations during training. As shown in Figure 2a, the proposed method, which combines time-stepping with active learning, achieves a higher level of accuracy of the state space compared to the pure time-stepping scheme, while converging substantially faster. The improvement in performance arises from the active learning component, which adaptively concentrates training points in economically relevant regions. In this model, the most critical region lies near the free boundary, where the solution exhibits sharp changes. As illustrated in Figure 2b, the sampling mechanism allocates more points in this neighborhood, improving local approximation quality and stabilizing convergence. A further comparison with the baseline neural network approach without time-stepping or active learning is shown in Figure A.2 in the appendix. In particular, our method successfully captures the endogenous jump in price returns at the free boundary, whereas the baseline network fails to reproduce this feature.

3.2 Optimal stopping

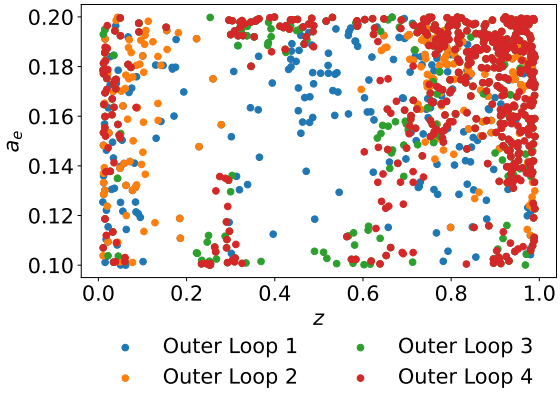
In this section, we solve an optimal stopping problem that follows Décamps, Gryglewicz, Morellec and Villeneuve (2017) closely. We provide a sketch of the model here and relegate the details to the appendix. Time $t \in [0, \infty)$ is continuous and there is a representative

Figure 2: Free boundary model loss progression and sampled points. Our method's total losses are 2.43×10^{-6} in financing-constraint region, 1.16×10^{-7} in shorting-constraint region, and 3.85×10^{-8} in no-constraint region.

(a) HJB error



(b) Active sampled points



firm with cash flows following an exogenous stochastic process $dX_t = \alpha A_t dt + \sigma_X A_t dW_t^X$, where A_t is the productivity that follows a Geometric Brownian Motion process $dA_t = \mu A_t dt + \sigma_A A_t dW_t^P$. The firm faces a fixed cost to raise equity, and there is no debt. That means that the cash holdings, denoted by c_t, becomes a state variable. The firm chooses the optimal cash holdings $c^* \in [0, 1]$ threshold below which it retains cash, and above which it issues dividends. The optimal cash holding boundary c^* is endogenous and depends on the value of the firm F_t. The optimization problem and the HJB problem is provided in Appendix B. A key distinction between this model and the free boundary model is that the boundary in this problem satisfies a smooth-pasting condition. In particular, the value function is continuously differentiable across c^*: both the function value and its relevant derivatives match on either side of the free boundary. We use this example to further demonstrate *residual-based* active learning.

A straightforward neural-network formulation parameterizes the value function F using an MLP and treats c^* as a single learnable scalar, optimized jointly with F via gradient

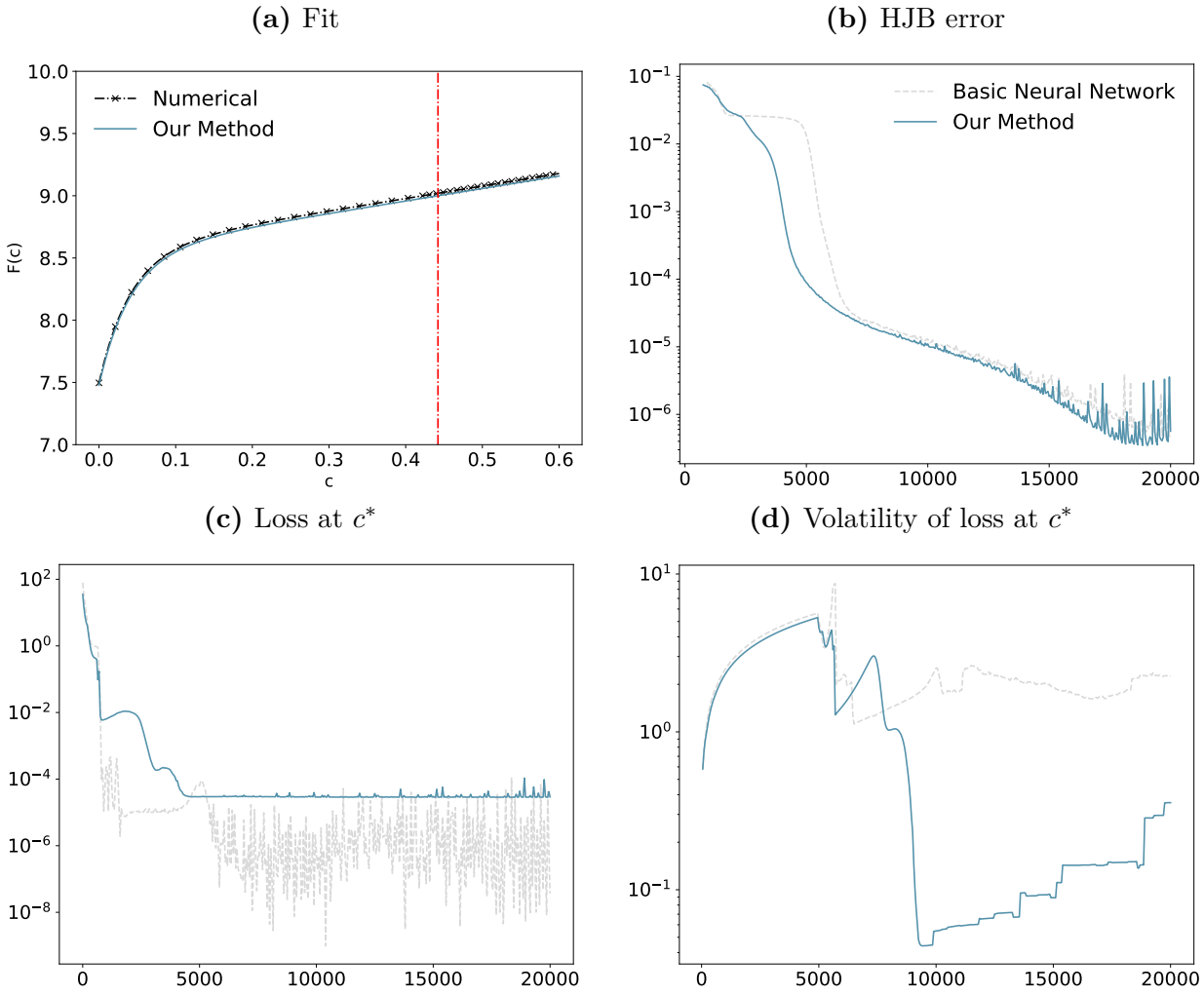
descent. In the experiments, we parametrize F as a 4-layer MLP, with 30 neurons per layer and tanh activated. The network is trained for 20,000 epochs using the Adam optimizer with a learning rate of 10^{-3}. However, as shown in Figures 3c and 3d, this strategy, where the optimality condition is enforced only at the single point c^* (gray dashed line), causes the loss to oscillate by 4-5 orders of magnitude. This behavior is driven by instability in the gradient computation at the free boundary. To mitigate this issue, we sample more training points in a neighborhood of c^* to better capture the local structure of the free boundary. We consider two complementary strategies, both in the spirit of *residual-based* active learning, which concentrate sampling in regions where approximation errors are the most unstable. First, at every 100 epoch, resample 50 points from a normal distribution $\mathcal{N}(c^*, 0.05)$ and add them to the training set, thereby concentrating the learning of the HJB equation near the free boundary. Second, instead of evaluating the optimality loss at a single point c^*, we compute it over a small band $[c^*-\delta, c^*+\delta]$ with $\delta = 0.02$ in our experiment, using samples drawn from the uniform probability measure μ. To emphasize accuracy near the free boundary while maintaining smoothness, we apply a triangular kernel weight $1 - \frac{|c-c^*|}{\delta}$. Define $l(c) = r - \mu$ and $m(c) = \alpha + c(r - \lambda - \mu)$, following the notation of the HJB equation in Appendix B. At the free boundary, the smooth-pasting conditions $F'(c^*) = 1$ and $F''(c^*) = 0$ imply $F(c^*) = m(c^*)/l(c^*)$. The banded optimality loss extends this condition to a neighborhood of c^*:

$$\text{MSE} = \frac{1}{\mu([c^* - \delta, c^* + \delta])} \int_{c^*-\delta}^{c^*+\delta} \left(1 - \frac{|c - c^*|}{\delta}\right) \left(\left| F(c) - \frac{m(c)}{l(c)} \right|^2 + |F'(c) - 1|^2 \right) d\mu(c).$$

With this active learning scheme, the resulting fit and the out-of-sample loss trajectories

are shown by the blue curves in Figure 3. While the final loss level is comparable to that obtained with the baseline neural-network approach, the training dynamics are substantially more stable, especially around the free boundary c^*. This improved stability makes it safer and more reliable to select the model achieving the lowest out-of-sample loss.

Figure 3: Benchmark optimal stopping problem. We set $\mu = 0.01$, $\sigma_A = 0.25$ as in the original paper. The MSE of F is 4.11×10^{-4} and MAE of the found free boundary c^* is 0.02. In (a) the red dash-dot line is the fitted c^*



Next, we extend the model to higher dimensions, by introducing a shock to the growth

rate of asset productivity, following Ornstein-Uhlenbeck process:

$$d\mu_t = a(b - \mu_t)dt + sdW_t^{\mu},$$

where dW_t^{μ} is orthogonal to dW_t^P and dW_t^T (the two stochastic processes in the benchmark model). The differential operator $\mathcal{L}$ acting on the value function $F(c, \mu)$ and the associated HJB residual operator $\mathcal{R}$ are given by

$$\begin{aligned}
\mathcal{L}[F](c,\mu) &= (\alpha + c(r - \lambda - \mu))F_c(c,\mu) + \frac{1}{2}(\sigma_A^2 c^2 - 2\rho\sigma_A\sigma_X c + \sigma_X^2)F_{cc}(c,\mu) \\
&\quad + a(b-\mu)F_{\mu}(c,\mu) + \frac{1}{2}s^2 F_{\mu\mu}(c,\mu) \\
\mathcal{R}[F](c,\mu) &= \mathcal{L}[F](c,\mu) - (r-\mu)F(c,\mu)
\end{aligned}$$

In this setting, the free boundary becomes state-dependent: the optimal threshold $c^*(\mu)$ varies with μ. The corresponding optimality (smooth-pasting) conditions are:

$$F_c(c^*(\mu),\mu) = 1, \quad F_{cc}(c^*(\mu),\mu) = 0$$

Therefore, the final system is

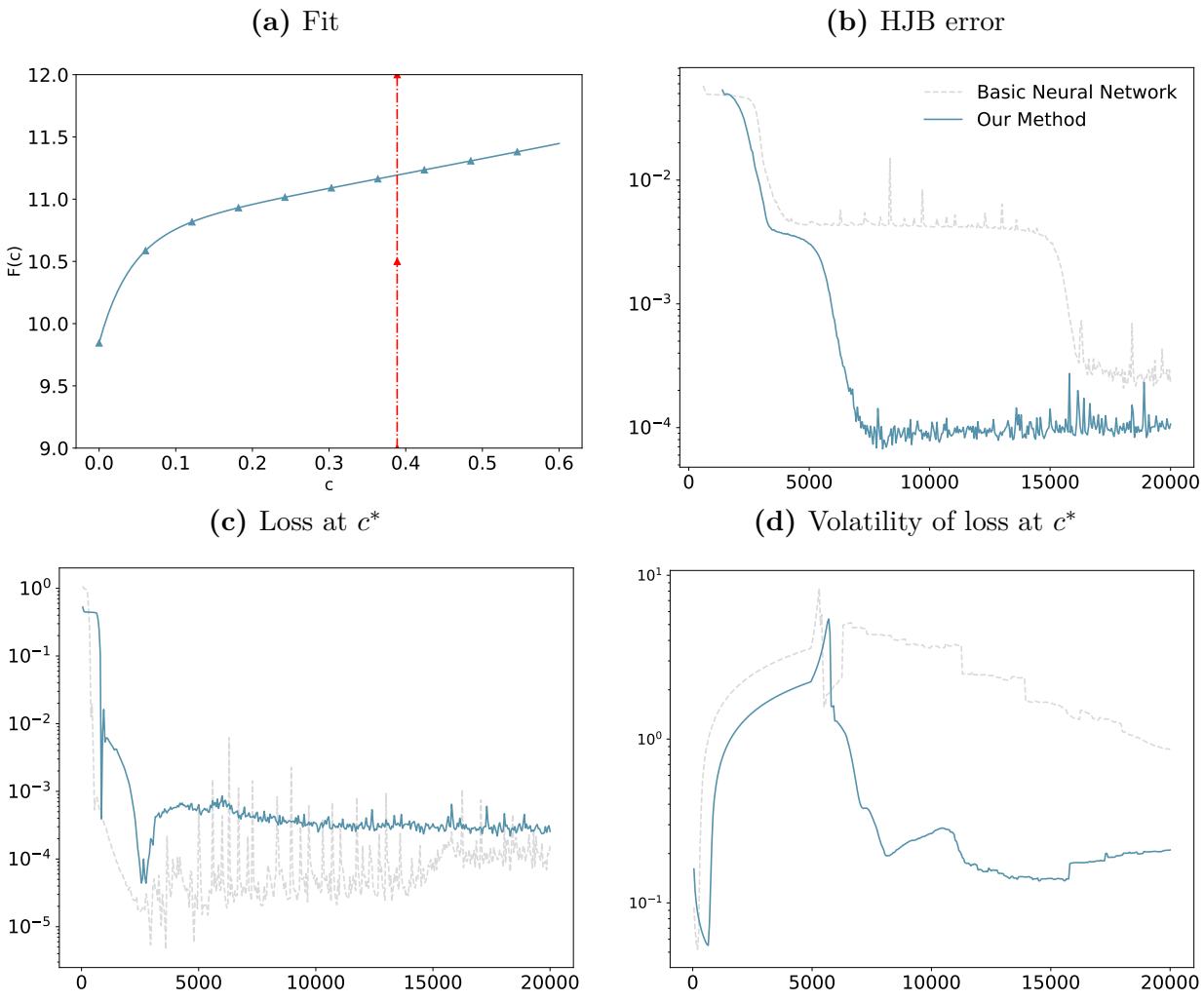
$$\begin{aligned}
&\mathcal{R}[F](c,\mu) = 0 \\
&F(0,\mu) = G(\mu) := \max\left\{ \sup_{c\in[0,1]} \left(F(c,\mu) - p(c+\phi)\right), \frac{\omega\alpha}{r-\mu} \right\} \\
&F_c(c^*(\mu),\mu) = 1, \quad F_{cc}(c^*(\mu),\mu) = 0
\end{aligned}$$

This extended model is more challenging to learn, as the second-order condition $F_{cc} = 0$ must be enforced along a one-dimensional manifold $(c^*(\mu), \mu)$ in the state space. To improve numerical stability, we impose an additional global structural constraint $F_{cc} \leq 0$ in the continuation region, which reflects concavity and regularizes the learning problem. The non-strict inequality accommodates the smooth-pasting condition $F_{cc}(c^*(\mu), \mu) = 0$ at the free boundary, while the constraint is enforced softly during training to allow $F_{cc} \approx 0$ in a neighborhood of c^*. Our active learning strategy mirrors the benchmark case: we oversample training points in the neighborhood of the free boundary $(c^*(\mu), \mu)$, and evaluate the optimality conditions over a small band around $c^*(\mu)$, using a triangular kernel along the c-direction at each μ. The resulting fits and loss trajectories are reported in Figure 4. Consistent with the benchmark model results, *residual-based* active learning near the free boundary and enforcing the optimality conditions in a neighborhood of c^* significantly improves convergence, both for the HJB residual and for the smooth-pasting conditions at the free boundary.

3.3 Stochastic volatility model

Next, we solve a model that introduces complexity in terms of having a coupled system of PDEs and algebraic equations with a highly non-linear value function throughout the state space. The presence of idiosyncratic state and concentrated risk makes these functions nonlinear. Moreover, there are multiple components in the loss function, including a market clearing condition. Adding a risky capital market clearing condition in the loss is prone to instabilities (see, for example, Azinovic and Zemlicka (2023)) and in general makes it

Figure 4: Optimal stopping with productivity shock. We set $\sigma_A = 0.25$. In (a), we plot the function $F(c)$ and $c^*(\mu)$ along $\mu = 0.01$.



difficult for neural networks to converge to the correct equilibrium function. To address this problem, we turn to the *loss weight-based* method where the algorithm actively allocates different weights to the components of the loss function depending on the trajectory of the loss over previous iterations. For example, if the algorithm struggles to learn the market clearing condition consistently in prior iterations, then a loss weight-based method allocates higher weight to this particular loss component in the current iteration. Similarly, if the algorithm found it easier to learn the clearing condition, then it attaches a lower weight.

This is active in the sense that the aggregate loss function dynamically adapts the weights and guides the learning process to reach the correct equilibrium solution. To show this concretely in action, we apply this to the stochastic volatility model that contains several components in the total loss - a) consumption first order condition, b) capital market clearing condition, c) portfolio first order condition, d) HJB equation of experts, and e) HJB equation of households. The parameters can be found in Appendix C, while we refer the readers to Di Tella (2017) for the detailed setup of the model. We parameterize the value functions ξ, ζ and endogenous variables p, r using neural networks. Each neural network contains 4 hidden layers with 30 neurons, activated by a *tanh* function. The final layer of ξ, ζ, p is restricted to be positive using a *SoftPlus* activation function. The learning rate is set to 0.001 for all models. A total of 500 points are randomly sampled in each time step for training. For the basic neural network method, we train for 20000 epochs; the loss plateaus after approximately 18,000 epochs, after which no further accuracy gains are observed. For our method with the time-stepping scheme, we train for 5000 epochs in each time step and train for 70 time iterations.

Figures 5a and 5c report the evolution of the total validation loss and the HJB error for the experts evaluated at the iterations corresponding to the running minimum of the total loss (the corresponding HJB error for households exhibits analogous behavior). Overall, both the basic time-stepping method and our *loss weight-based* approach converge to similar final loss, and both outperform the basic neural network baseline. Figure 5b presents the dynamic loss weights during the first time-step iteration. We can see that our method automatically assigns a higher weight to the market clearing condition at the beginning of the training process. As the market clearing condition is satisfied in the later stages, the portfolio first-

order conditions are violated, thus receiving larger weights. After a sufficient number of iterations, the weights are close to each other. The loss weights typically do not add to one at convergence, becase the softmax in λ_i^{bal} normalizes the weights to sum to m (the number of loss components), and are associated with the variance of the loss values.[5] At the end of each time step, the loss weights are reset and the dynamic reweighting procedure is restarted for the subsequent iteration. Table 1 reports relative mean absolute errors (relative MAEs), defined as $\frac{E[|y-\hat{y}|]}{E[|y|]}$, where y is the finite-difference solution and $\hat{y}$ is the neural network solutions. For nearly all variables, our combined approach, time stepping with *loss weight-based* active learning, achieves the lowest relative errors. Figure 5d shows the fit of value functions.

	p	σ_x	$\Omega = \xi/\zeta$	$\sigma + \sigma_p$	π	r	$\hat{e}$	$\hat{c}$
Basic	31.38%	6.27%	0.48%	15.83%	40.69%	21.84%	39.39%	39.29%
Time-stepping	4.72%	2.63%	1.31%	3.78%	4.26%	15.85%	6.61%	7.91%
Our Method	3.49%	2.84%	0.64%	2.21%	2.55%	16.33%	4.86%	5.47%

Table 1: Stochastic volatility model relative MAEs.

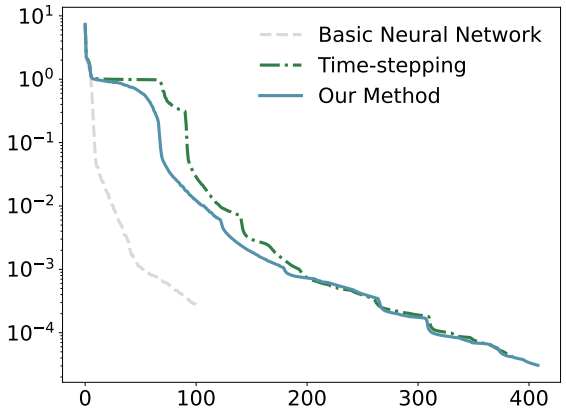
3.4 N-trees

In the final section, we solve a high-dimensional model with multiple risky assets, in the spirit of Martin (2013), but with CRRA utility. The full model specification is provided in Appendix D. The state variables lie on the simplex $\begin{cases} z_i \geq 0, \\ \sum_{i=1}^{N-1} z_i \leq 1 \end{cases}$, from which we draw the training samples. The idea of active learning is still applicable. In particular, we use the *residual-based* active learning method to focus on the regions that are computationally more

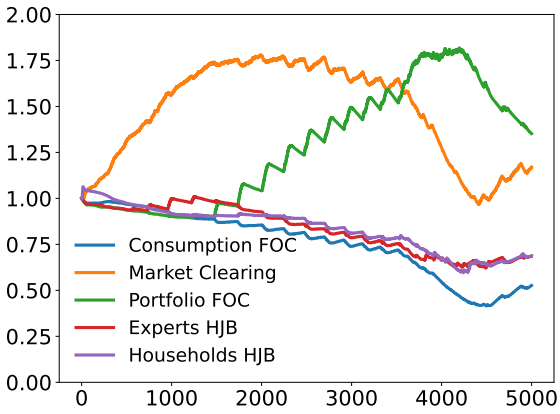
[5] See, for example, Bischof and Kraus (2025) and Cipolla, Gal and Kendall (2018).

Figure 5: Stochastic volatility model loss progression, loss weights and fitted value function ratios.

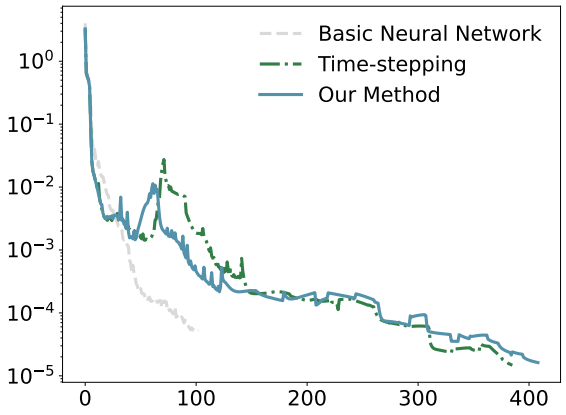
(a) Total validation loss



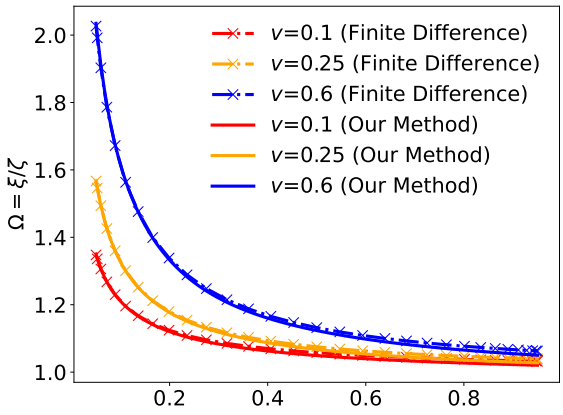
(b) Loss weights variation



(c) HJB (Expert) error



(d) Value function



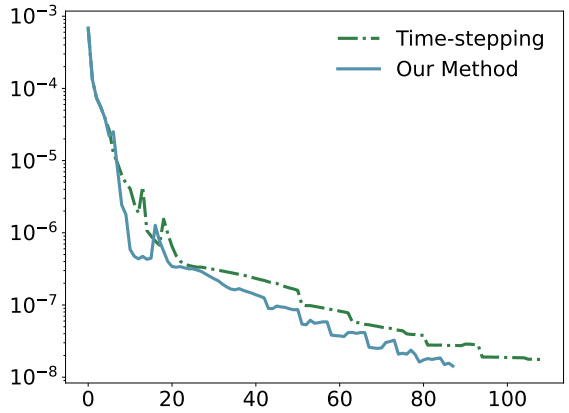
problematic. That is, we add to the training data sampled from the simplex with points sampled from the regions that have violated equilibrium conditions in previous iterations, following Algorithm 2.

We solve a 50-tree model, where each tree is an exogenous stochastic process driven by independent Brownian shocks. We denote κ_i and q_i to be the value function and price of tree i respectively. Following Han et al. (2021), we use the HJB equation error on out-of-sample datapoints to evaluate accuracy. The HJB residual serves as the continuous-time analogue of the Euler equation error widely used in discrete-time computational economics.

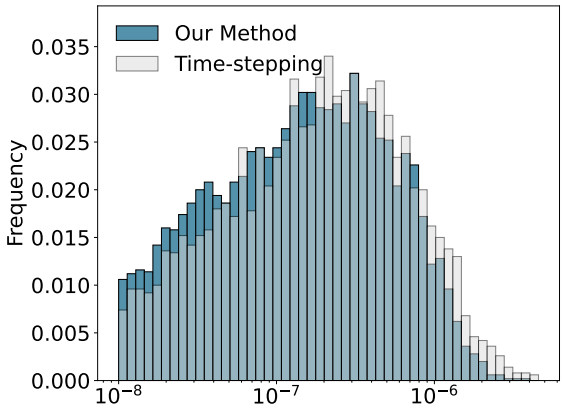
Figure 6a reports the time evolution of the aggregate HJB error for the economy evaluated at the iterations corresponding to the running minimum of the total loss. As in the previous examples, our approach attains comparable loss levels while requiring fewer training epochs. Moreover, Figure 6b illustrates the distribution of pointwise HJB residuals (squared error of HJB equation, whose mean value is the model HJB error) using the best models, showing that *residual-based* learning shifts the out-of-sample residual mass toward smaller magnitudes, thereby improving overall solution accuracy.

Figure 6: 50-tree model performance. The distribution of pointwise HJB residuals is computed over 5000 random out-of-sample datapoints.

(a) HJB loss



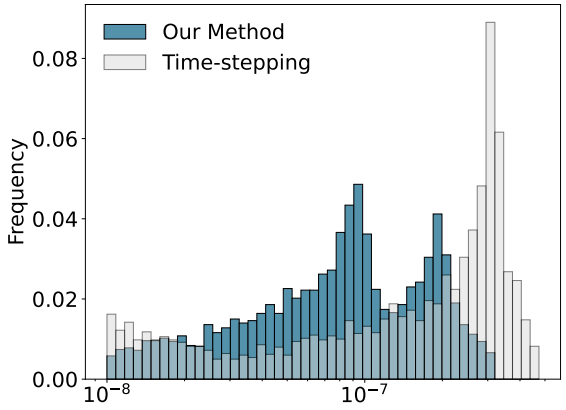
(b) Distribution of pointwise HJB residuals



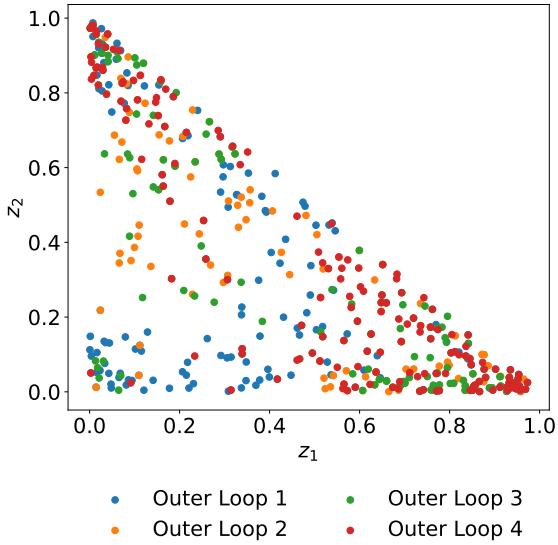
Although the 50-dimensional example demonstrates that active learning accelerates convergence and yields more accurate approximations, it does not explain why this improvement occurs. We provide some insights into this using a 3-tree model. Figure 7a presents the distribution of HJB residuals with and without active sampling. Active learning reduces errors across a wide range of datapoints, effectively shifting the entire residual distribution toward smaller magnitudes. Figure 7b displays active training points over time iterations. The blue dots represent the training points at the first time iteration. They are uniformly

Figure 7: 3-tree model performance. The distribution of pointwise HJB residuals is computed over 5000 random out-of-sample datapoints. In (c) and (d), κ_1 and q_1 are plotted against z_1 with $z_2 = 0.2$ fixed.

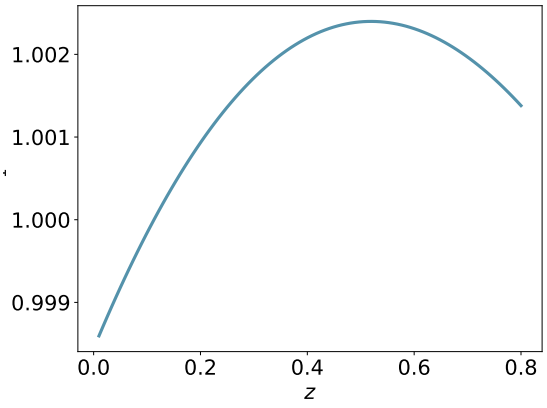
(a) HJB Error



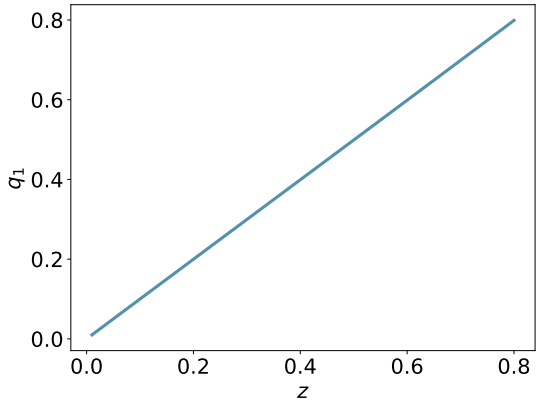
(b) Active sampled points



(c) κ_1



(d) q_1

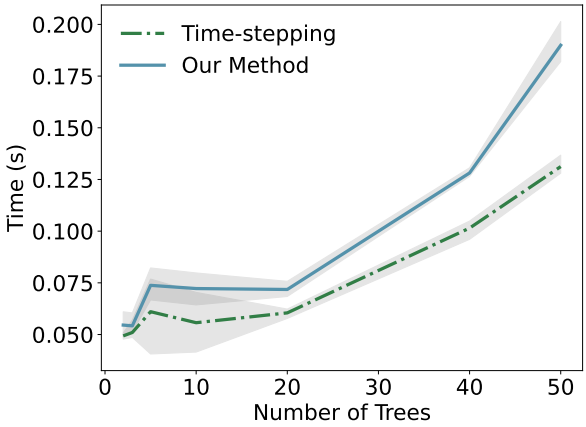


distributed in the simplex of the state variables. The orange dots are the points that are sampled from the regions that have violated the equilibrium conditions in the second iteration. As training proceeds, active points concentrate near the boundaries of the state space, where the approximation errors are largest, highlighting how active sampling adaptively targets the most challenging regions.

3.4.1 Training time and memory analysis

In this section, we analyze the training time and memory required for the N-tree models. We solve a total of 6 models for $N = 2, 3, 5, 10, 20, 50$. We solve the model using the time-stepping method, and the time-stepping method with active learning (our method). The epoch training time includes the time for sampling, the forward pass of the neural network, loss computation, the backward propagation of the gradient, and the optimization step. The experiments are performed on Google Colab A100 GPU. Figure 8 shows the average training time per epoch for each N-tree model. The mean is plotted as lines, and the gray area shows the $[5\%, 95\%]$ region of the runtime. First, we observe that the training time does not increase significantly as the number of trees grows large. This exemplifies the power of using neural networks as emphasized by many in the literature. The second thing to note is that the per epoch time taken for our method is similar to the basic neural network method. This shows that our active learning methods can improve the accuracy without increasing too much computational overhead.

Figure 8: N-tree model training time per epoch. Training is performed over 10 time steps, with 1,000 epochs per time step, resulting in a total of 10,000 epochs. For the 50-dimensional model, this corresponds to a total training time of approximately 30 minutes.



	CUDA Memory (MB)						FLOPS ($\times 10^9$)					
	2-Tree	3-Tree	5-Tree	10-Tree	20-Tree	50-Tree	2-Tree	3-Tree	5-Tree	10-Tree	20-Tree	50-Tree
Time-stepping	56.75	82.69	155.61	455.28	1654.78	10871.23	2.77	4.34	8.61	26.38	95.88	653.75
Our Method	120.01	189.50	378.68	1166.70	4271.01	28402.33	4.03	6.32	12.56	38.46	139.66	

Table 2: N-Tree Model Memory Consumption and FLOPs.

Table 2 shows the total memory usage and floating point operations (FLOPs) for a single full training iteration of each model, using a batch size of 200 training points. A full iteration includes sampling stage, forward pass of the neural network, loss value computation, backward gradient propagation, and optimization. For our *residual-based* active learning, we first generate 1,000 random points and compute their residuals, then select the top $k = 40$ points with the highest residual. In principle, one could resample candidate points multiple times to increase exploration, but in our experiments 1,000 candidates are sufficient. This resampling is repeated 5 times over the 1,000 epochs within each time step. The memory usage and FLOPs are computed using PyTorch's profiler, which has a tendency to overestimate the usage due to the additional computation from tracking done by the profiler.[6] Because the profiler itself increases computational overhead, the experiment can run out of memory on large models. Accordingly, we report FLOPs only for models up to $N = 20$ and $N = 50$ for with and without active learning, respectively. While our method incurs higher memory and FLOPs due to the additional 1,000 points for residual computation, the total increase is modest: even with roughly five times more computation, memory and FLOPs rise by only a factor of 2–3 compared to the baseline.

[6]See `https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html`.

4 Conclusion

This paper introduces Active Learning Inspired Equilibrium Nets (ALIENs), a neural network-based method for solving equilibrium models in continuous-time economics. By combining time-stepping and active learning, ALIENs extend the existing deep learning based methods by transforming the economic model into sequences of contraction mappings to ensure stability and convergence in the numerical procedure. We demonstrate our method using a few applications in macro-finance, corporate finance, and asset pricing, and show that it improves accuracy and convergence by focusing computational efforts on economically relevant regions. The accompanying *Deep-Macrofin+* library further supports the adoption of these techniques. Our method offers a robust framework for solving complex economic models, paving the way for future exploration of higher-dimensional problems and new neural network architectures.

ALIENs for Continuous Time Economies

Appendix

The first few sections of this appendix present implementation details, parameter configurations, and additional results for the models analyzed in the main text. The optimization problems underlying those models are provided in the online appendix. Section E of this appendix contains the proofs of all propositions stated in the paper.

A Free boundary models

A.1 Benchmark

This section reports the parameters and system setup for the one-dimensional free boundary model. The model is based on Brunnermeier and Sannikov (2014). The state variable is experts' share of wealth: $\eta \in (0, 1)$. For implementation, we use $[0.01, 0.99]$.

Parameter	Definition	Value
σ	exogenous volatility of capital	0.1
δ_e	depreciation rate of capital for experts	0.05
δ_h	depreciation rate of capital for households	0.05
a	productivity of experts	0.11
a_h	productivity of households	0.07
ρ	discount rate of experts	0.06
r	discount rate of households	0.05
κ	adjustment cost parameter	2

Table A.1: Benchmark free boundary model parameters

We solve for the following system of PDEs when $\psi < 1$:

$$(r(1-\eta)+\rho\eta)q = \psi a_e + (1-\psi)a_h - \iota$$

$$\sigma_t^q + \sigma = \frac{\sigma}{1 - \frac{1}{q}\frac{\partial q}{\partial \eta}(\psi - \eta)}$$

$$(\sigma + \sigma_t^q)^2 \frac{q(\psi-\eta)}{\eta(1-\eta)} = a_e - a_h,$$

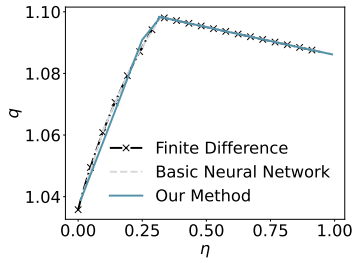
and the following PDE when $\psi = 1$:

$$(r(1-\eta)+\rho\eta)q = a_e - \iota$$

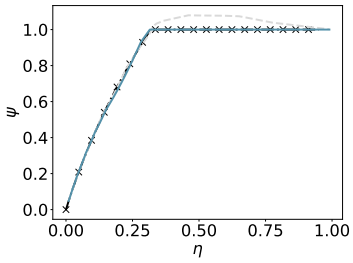
$$\sigma_t^q + \sigma = \frac{\sigma}{1 - \frac{1}{q}\frac{\partial q}{\partial \eta}(\psi - \eta)}.$$

Figure A.1: Benchmark free boundary model equilibrium plots (ReLU activation). The MSEs for the split method are 1.50×10^{-6} for price, 4.11×10^{-5} for experts capital shares, and 1.93×10^{-6} for price return diffusion.

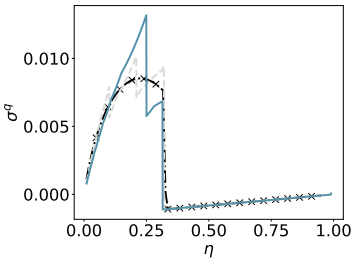
(a) Price



(b) Capital share: experts



(c) Price return diffusion



A.2 Free boundary model with additional shocks

Details of the model are provided in Gopalakrishna (2020). State Variables are 1) Experts' share of wealth: $z \in (0,1)$ and 2) Productivity of experts: $a_e \in (fl, fu)$. For implementation, we use $[0.01, 0.99] \times [0.1, 0.2]$ (Therefore $\hat{a_e} = \frac{fl+fu}{2} = 0.15$). Agent value functions are J_e

for experts and J_h for households. Endogenous variables are capital share of expert ψ, price q, and expert's inside equity share χ.

Parameter	Definition	Value
a_h	productivity of households	0.03
σ	exogenous volatility of capital	0.1
δ	depreciation rate of capital	0.05
p	persistent parameter for a_e	0.01
v	variation parameter for a_e	2.5
$\hat{a_e}$	mean of a_e	0.15
fl	min of a_e	0.1
fu	max of a_e	0.2
ϕ	correlation between dZ_t^k and dZ_t^a	0.5
γ	risk aversion	5
ρ	discount rate	0.05
κ	adjustment cost parameter	5
λ_d	birth/death rates of agents	0.03
$\bar{z}$	portion of experts born	0.1
$\underline{\chi}$	maximum allowed equity	1

Table A.2: Free boundary model parameters

Let $j \in \{e, h\}$ index the agents. The model follows from Brunnermeier and Sannikov (2016) with Epstein Zin utility and an additional state variable $a_{e,t}$ denoting productivity of experts. Taking the additional state variable into account leads to the following growth rate equations:

$$\sigma^{q,k,1} = \sigma + \sigma^{q,k}$$

$$\begin{aligned}\mu^z = &\frac{a_e - \iota}{q} - \hat{c}_e \\ &+ (\theta_e - 1)(\sigma^{q,k,1}(\zeta_e^1 - \sigma^{q,k,1}) + \sigma^{q,a}(\zeta_e^2 - \sigma^{q,a}) - 2\phi\sigma^{q,k,1}\sigma^{q,a}) \\ &+ (1 - \underline{\chi})(\sigma^{q,k,1}(\zeta_e^1 - \zeta_h^1) + \sigma^{q,a}(\zeta_e^2 - \zeta_h^2)) + \frac{\lambda_d}{z}(\bar{z} - z)\end{aligned}$$

$$\mu^q = \frac{1}{q}\left(\frac{\partial q}{\partial z}\mu^z z + \frac{\partial q}{\partial a_e}\mu_{ae} + \frac{1}{2}\frac{\partial^2 q}{\partial z^2}z^2((\sigma^{z,k})^2 + (\sigma^{z,a})^2 + 2\phi\sigma^{z,k}\sigma^{z,a})\right)$$
$$+ \frac{1}{q}\left(\frac{1}{2}\frac{\partial^2 q}{\partial a_e^2}\sigma_{ae}^2 + \frac{\partial^2 q}{\partial z \partial a_e}(\phi\sigma^{z,k} + \sigma^{z,a})\sigma_{ae} z\right)$$

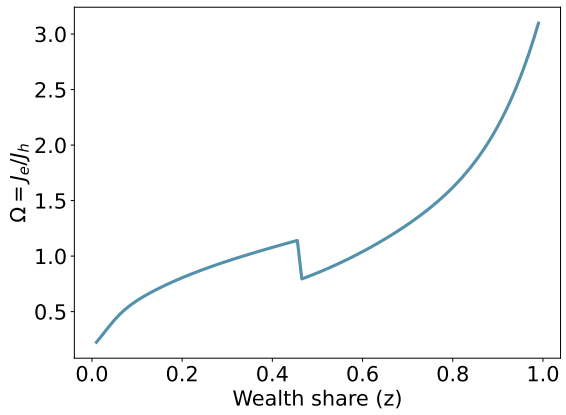
$$\mu_j^J = \frac{\gamma}{2}((\sigma_j^{J,k})^2 + (\sigma_j^{J,a})^2 + 2\phi\sigma_j^{J,k}\sigma_j^{J,a} + \sigma^2) - (\Phi - \delta)$$
$$+ (\gamma - 1)(\sigma_j^{J,k}\sigma + \phi\sigma\sigma_j^{J,a}) - \rho(\log(\rho) - \log(J_j) + \log(zq))$$

$$\mu_j^R = \frac{a_j - \iota}{q} + \Phi - \delta + \mu^q + \sigma\sigma^{q,k} + \phi\sigma\sigma^{q,a}$$

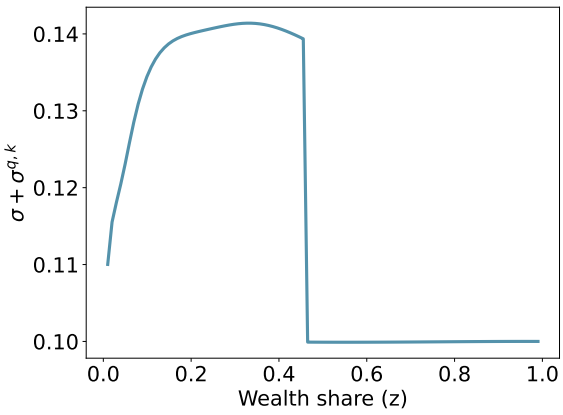
$$\mu_j^J J_j = \frac{\partial J_j}{\partial t} + \frac{\partial J_j}{\partial z}\mu^z z + \frac{\partial J_j}{\partial a_e}\mu_{ae} + \frac{1}{2}\frac{\partial^2 J_j}{\partial z^2}z^2\left((\sigma^{z,k})^2 + (\sigma^{z,a})^2 + 2\phi\sigma^{z,k}\sigma^{z,a}\right)$$
$$+ \frac{1}{2}\frac{\partial^2 J_j}{\partial a_e^2}\sigma_{ae}^2 + \frac{\partial^2 J_j}{\partial z \partial a_e}(\phi\sigma^{z,k} + \sigma^{z,a})\sigma_{ae} z$$

Figure A.2: Free boundary model equilibrium plots. In these plots, we fix $a_e = 0.15$

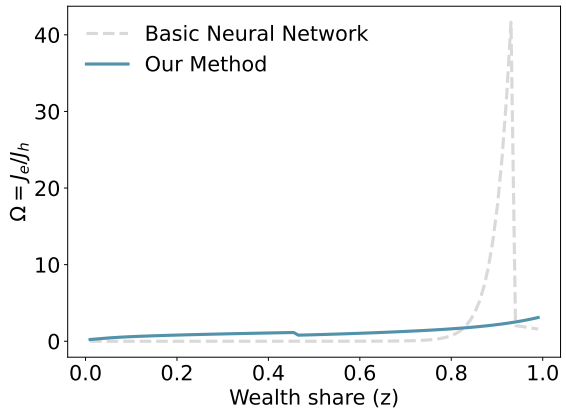
(a) $\Omega = J_e/J_h$



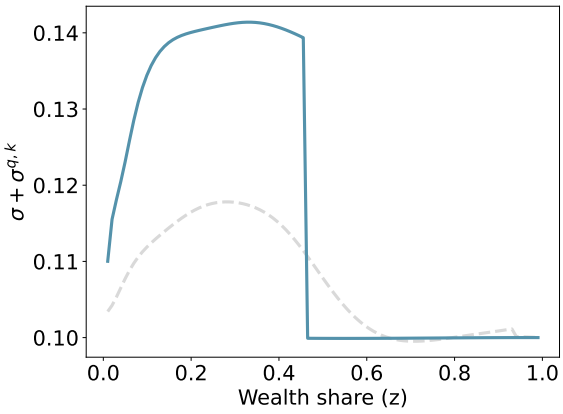
(b) $\sigma + \sigma^{q,k}$



(c) $\Omega = J_e/J_h$ (comparison)



(d) $\sigma + \sigma^{q,k}$ (comparison)



B Optimal stopping

B.1 Benchmark

The productivity of assets follow

$$dA_t = \mu A_t dt + \sigma_A A_t dW_t^P$$

In addition to the permanent shocks, cash flows are subject to short-term shocks. The operating cash flows are given by

$$dX_t = \alpha A_t dt + \sigma_X A_t dW_t^X,$$

where the correlation between dW_t^P and dW_t^X is $\rho \in [-1, 1]$

The dynamics of cash flows can be rewritten as

$$dX_t = \alpha A_t dt + \sigma_X A_t \left(\rho dW_t^P + \sqrt{1-\rho^2} dW_t^T \right),$$

where W^T is independent from W^P.

Let c denote the cash reserve scaled by earnings and $F(c)$ denote the value of firm equity. The evolution of c is

$$dC_t = (\alpha + C_t(r - \lambda - \mu))dt + \sigma_X \sqrt{1-\rho^2} dW_t^T + (\rho\sigma_X - C_t\sigma_A) d\tilde{W}_t^P + \frac{dE_t}{pA_t} - \frac{d\phi_t + dL_t}{A_t},$$

where E_t is external financing, ϕ_t is fixed cost and L_t is dividend.

The value function satisfies:

$$(r-\mu)F(c) = (\alpha + c(r-\lambda-\mu))F'(c) + \frac{1}{2}(\sigma_A^2 c^2 - 2\rho\sigma_A\sigma_X c + \sigma_X^2)F''(c), c < c^*$$

$$F(c) = F(c^*) + c - c^*, c \geq c^*$$

$$F(0) = \max\left\{\max_{c\in[0,\infty]}(F(c) - p(c+\phi)), \frac{\omega\alpha}{r-\mu}\right\}$$

$$F'(c^*) = 1 \quad F''(c^*) = 0$$

Parameter	Definition	Value
c	Scaled cash holdings	$c \in [0,1]$ (problem domain)
c^*	Target cash holdings	TBD as free boundary
μ	Growth rate of asset productivity	$\mu = 0.01$
σ_A	Volatility of permanent shocks	$\sigma_A = 0.25$
σ_X	Volatility of short-term shocks	$\sigma_X = 0.12$
ρ	Correlation between shocks	$\rho = -0.2$
λ	Carry cost of liquidity	$\lambda = 0.02$
α	Mean rate of cash flows	$\alpha = 0.18$
ϕ	Fixed financing cost	$\phi = 1.002$ (liquidation)
p	Proportional financing cost	$p = 1.06$
ω	Asset liquidation-value ratio	$\omega = 0.55$
r	Risk-free rate	$r = 0.03$

Table A.3: Benchmark optimal stopping model parameters

B.2 Model with additional shock

Assume that the growth rate of asset productivity also experiences a shock following Ornstein-Uhlenbeck process:

$$d\mu_t = a(b-\mu_t)dt + sdW_t^\mu,$$

s.t. dW^μ is orthogonal to dW^P and dW^T, so we skip the cross-terms.

Now the system of SDEs is:

$$
\begin{aligned}
d\mu_t &= a(b-\mu_t)dt + sdW_t^{\mu} \\
dA_t &= \mu_t A_t dt + \sigma_A A_t dW_t^P \\
dX_t &= \alpha A_t dt + \sigma_X A_t \left(\rho dW_t^P + \sqrt{1-\rho^2} dW_t^T\right) \\
dC_t &= (\alpha + C_t(r-\lambda-\mu_t))dt + \sigma_X\sqrt{1-\rho^2}dW_t^T + (\rho\sigma_X - C_t\sigma_A)d\tilde{W}_t^P + \frac{dE_t}{pA_t} - \frac{d\phi_t + dL_t}{A_t}
\end{aligned}
$$

For each μ, the boundary condition is:

$$
F(0,\mu) = \max\left\{\sup_{c\in[0,1]}\left(F(c,\mu) - p(c+\phi)\right), \frac{\omega\alpha}{r-\mu}\right\}
$$

The differential operator $\mathcal{L}$ acting on $F(c,\mu)$ and the corresponding HJB residual operator $\mathcal{R}$ are:

$$
\begin{aligned}
\mathcal{L}[F](c,\mu) &= (\alpha + c(r-\lambda-\mu))F_c(c,\mu) + \frac{1}{2}(\sigma_A^2 c^2 - 2\rho\sigma_A\sigma_X c + \sigma_X^2)F_{cc}(c,\mu) \\
&\quad + a(b-\mu)F_{\mu}(c,\mu) + \frac{1}{2}s^2 F_{\mu\mu}(c,\mu) \\
\mathcal{R}[F](c,\mu) &= \mathcal{L}[F](c,\mu) - (r-\mu)F(c,\mu)
\end{aligned}
$$

The threshold $c^*(\mu)$ is now μ-dependent. The optimality conditions are:

$$
F_c(c^*(\mu),\mu) = 1, \quad F_{cc}(c^*(\mu),\mu) = 0
$$

Therefore, the final system is

$$\mathcal{R}[F](c,\mu) = 0$$

$$F(0,\mu) = G(\mu) := \max\left\{ \sup_{c\in[0,1]} \left(F(c,\mu) - p(c+\phi)\right), \frac{\omega\alpha}{r-\mu} \right\}$$

$$F_c(c^*(\mu),\mu) = 1, \quad F_{cc}(c^*(\mu),\mu) = 0$$

Parameter	Definition	Value
c	Scaled cash holdings	$c \in [0,1]$ (problem domain)
μ	Growth rate of asset productivity	$\mu \in [0.0, 0.02]$ (problem domain, smaller than r)
a	OU process rate	$a = 1$
b	OU process mean	$b = 0.01$
s	OU process std	$s = 0.01$
c^*	Target cash holdings	$c^*(\mu)$ TBD as free boundary
σ_A	Volatility of permanent shocks	$\sigma_A = 0.25$
σ_X	Volatility of short-term shocks	$\sigma_X = 0.12$
ρ	Correlation between shocks	$\rho = -0.2$
λ	Carry cost of liquidity	$\lambda = 0.02$
α	Mean rate of cash flows	$\alpha = 0.18$
ϕ	Fixed financing cost	$\phi = 1.002$ (liquidation)
p	Proportional financing cost	$p = 1.06$
ω	Asset liquidation-value ratio	$\omega = 0.55$
r	Risk-free rate	$r = 0.03$

Table A.4: Optimal stopping model parameters

C Stochastic volatility model

This model follows Di Tella (2017), and we refer the readers to the original paper for more details. Here, we only provide the set of parameters used.

Parameter	Definition	Value
a	productivity	1
σ	volatility of TFP shocks	0.0125
λ	mean reversion coefficient for idiosyncratic risk	1.38
$\bar{v}$	long-run mean of idiosyncratic risk	0.25
$\bar{\sigma_v}$	idiosyncratic volatility of capital on aggregate risk	-0.17
ρ	discount rate	0.0665
γ	risk aversion rate	5
ψ	inverse of elasticity of intertemporal substitution	$\psi = 0.5$ s.t. EIS=2
τ	Poisson retirement rate for experts	1.15
ϕ	moral hazard	0.2
A	second order coefficient for investment function	53
B	first order coefficient for investment function	-0.87
δ	shift for investment function	0.05

Table A.5: Stochastic volatility model parameters

Type	Definition
State Variables	$(x, v) \in [0.05, 0.95] \times [0.05, 0.95]$
Agents	ξ (experts), ζ (households)
Endogenous Variables	p (price), r (risk-free rate)

Table A.6: Stochastic volatility model variables

D N-trees

This model follows Martin (2013), but with CRRA utility. The state variables are the share of tree $i \in [1, N-1]$: $\begin{cases} z_i \geq 0, \\ \sum_{i=1}^{N-1} z_i \leq 1 \end{cases}$. Define $\mathbf{z} = (z_1, ..., z_{N-1})$, and $\tilde{\mathbf{z}} = \left(z_1, ..., z_{N-1}, 1 - \sum_{i=1}^{N-1} z_i\right)$. Similarly, we define the geometric drifts by $\mu^{\mathbf{z}} = (\mu^{z_1}, ..., \mu^{z_{N-1}})$, and $\mu^{\tilde{\mathbf{z}}} = (\mu^{z_1}, ..., \mu^{z_{N-1}}, \mu^{z_N})$. The geometric diffusions $\sigma^{\mathbf{z}}$ and $\sigma^{\tilde{\mathbf{z}}}$ and the arithmetic drifts and diffusions $\mu_a^{\mathbf{z}}, \mu_a^{\tilde{\mathbf{z}}}, \sigma_a^{\mathbf{z}}, \sigma_a^{\tilde{\mathbf{z}}}$ are defined similarly. We use $\cdot$ to denote the dot product and $\circ$ to denote the Hadamard (element-wise) product. The value functions are defined as a single neural network $\kappa : \mathbb{R}^{N-1} \to \mathbb{R}^N$, $z \mapsto \kappa(z)$.

Parameter	Definition	Value
γ	Household risk aversion	5
ρ	Fund discount rate	0.05
μ^{y_i}	Mean of the ith state variable	$0.01i$
σ^{y_i}	Std of the ith state variable	$0.01i$

Table A.7: N-tree parameters

We compute the prices $\mathbf{q} = \frac{\tilde{\mathbf{z}}}{\kappa}$ (component-wise).

State Dynamics: For $i = 1, ..., N-1$,,

$$\mu^{z_i} = \mu^{y_i} - \mu^y \cdot \tilde{\mathbf{z}} + \sigma^y \cdot \tilde{\mathbf{z}}(\sigma^y \cdot \tilde{\mathbf{z}} - \sigma^{y_i})$$

$$\sigma^{z_i} = \sigma^{y_i} - \sigma^y \cdot \tilde{\mathbf{z}}$$

The arithmetic drifts and diffusions of the first $N-1$ state variables are defined as $\mu_a^{\mathbf{z}} = \mu^{\mathbf{z}} \circ \mathbf{z}$

and $\sigma_a^{\mathbf{z}} = \sigma^{\mathbf{z}} \circ \mathbf{z}$. The N-th share follows from the adding-up constraint:

$$\mu_a^{z_N} = -\sum_{i=1}^{N-1} \mu_a^{z_i}, \quad \mu^{z_N} = \frac{\mu_a^{z_N}}{z_N}$$
$$\sigma_a^{z_N} = -\sum_{i=1}^{N-1} \sigma_a^{z_i}, \quad \sigma^{z_N} = \frac{\sigma_a^{z_N}}{z_N}$$

Using Ito's lemma, the price dynamics are:

$$\mu^{\mathbf{q}} = \frac{1}{\mathbf{q}} \circ \left(\nabla_{\mathbf{z}} \mathbf{q} \mu_a^{\mathbf{z}} + \frac{1}{2} \mathrm{Tr} \left(\sigma_a^{\mathbf{z}} (\sigma_a^{\mathbf{z}})^T \nabla_{\mathbf{z}}^2 q \right) \right)$$
$$\sigma^{\mathbf{q}} = \frac{1}{\mathbf{q}} \circ (\nabla_{\mathbf{z}} \mathbf{q} \sigma_a^{\mathbf{z}})$$

The risk-free rate and prices of risk are:

$$r = \rho + \gamma \mu^{\mathbf{y}} \cdot \tilde{\mathbf{z}} - \frac{1}{2} \gamma(\gamma+1)(\tilde{\mathbf{z}} \circ \tilde{\mathbf{z}}) \cdot (\sigma^{\mathbf{y}} \circ \sigma^{\mathbf{y}})$$
$$\zeta = \gamma \tilde{\mathbf{z}} \circ \sigma^{\mathbf{y}}$$

The value function follows the following dynamics:

$$\mu^{\kappa} = \mu^{\tilde{\mathbf{z}}} - \mu^{\mathbf{q}} + \sigma^{\mathbf{q}} \circ (\sigma^{\mathbf{q}} - \sigma^{\tilde{\mathbf{z}}})$$
$$\sigma^{\kappa} = \sigma^{\tilde{\mathbf{z}}} - \sigma^{\mathbf{q}}$$

The HJB equations are solved for each component of κ:

$$\mu^{\kappa} \circ \kappa = \nabla_t \kappa + \nabla_{\mathbf{z}} \kappa \mu_a^{\mathbf{z}} + \frac{1}{2} \mathrm{Tr} \left(\sigma_a^{\mathbf{z}} (\sigma_a^{\mathbf{z}})^T \nabla_{\mathbf{z}}^2 \kappa \right)$$

$$\sigma^{\kappa} \circ \kappa = \nabla_{\mathbf{z}} \kappa \sigma_a^{\mathbf{z}}$$

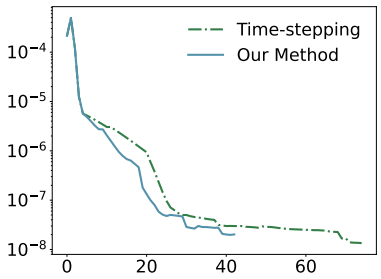
We parameterize κ as a multi-input, multi-output neural network with $N-1$ inputs and N outputs. Each neural network has 4 hidden layers with 80 neurons each, activated by *tanh*. Outputs are restricted to be positive using *SoftPlus*. Learning rates are set to 0.005 for all models. In each time step (or epoch without time-stepping), 200 points are randomly sampled for training. Models are trained for 1000 epochs in each time step and for upto 10 time iterations. Residual-based learning occurs 5 times at equal intervals within each time step or throughout the 1000 epochs *i.e.* every 200 epochs.

D.1 Benchmark 2-tree model

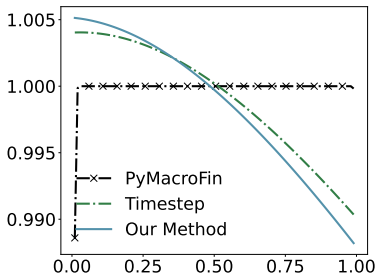
We train a simple 2-tree model using both a neural network approach and a finite difference method to validate our model setup. Figure A.3a shows the loss convergence of these models, comparing scenarios with and without residual-based learning. It is evident that the neural network model achieves faster convergence with residual-based sampling. Figure A.3b presents the equilibrium plots of the 2-Tree model using both finite difference and neural network approaches. The neural network approach, particularly with residual-based learning, provides the best approximation of the equilibrium distribution for κs and qs. Meanwhile, all models perform similarly for ζs and r.

Figure A.3: 2-tree HJB error progression and function approximations. The MSEs between our method and the numerical methods are 3.07×10^{-5} for κ and 1.49×10^{-5} for q

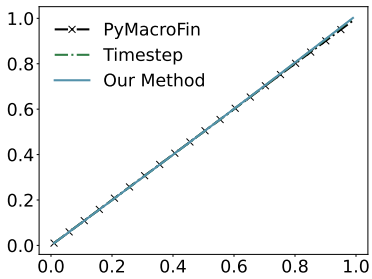
(a) HJB errors



(b) κ_1



(c) q_1



E Proofs of propositions

(i). Proposition 1

Proof. Fix $\varepsilon > 0$ sufficiently small so that f is continuous on $(x_0 - \varepsilon, x_0)$ and $(x_0, x_0 + \varepsilon)$ respectively. Let $a = f(x_0^-)$, $b = f(x_0^+)$.

By assumption, $\Phi_{\theta_n} \to f$ uniformly on compact subsets of $[0,1] \setminus \{x_0\}$. Hence, for any compact $K \subset [0,1] \setminus \{x_0\}$ $\int_K |\Phi_{\theta_n}(x) - f(x)|^2 \, d\mu(x) \to 0$.

Write $y_n = \Phi_{\theta_n}(x_0)$. By the uniform equicontinuity assumption, for every $\delta > 0$, there exists $\eta \in (0, \varepsilon)$, independent of n, such that $|x - x_0| < \eta$ implies $|\Phi_{\theta_n}(x) - y_n| < \delta$ for all n. For n large enough, continuity of f on $(x_0 - \eta, x_0)$ gives $|f(x) - a| < \delta$ for all $x \in (x_0 - \eta, x_0)$. Therefore, for $x \in (x_0 - \eta, x_0)$:

$$|\Phi_{\theta_n}(x) - f(x)| \geq |y_n - a| - |\Phi_{\theta_n}(x) - y_n| - |f(x) - a| \geq |y_n - a| - 2\delta.$$

Similarly, for $x \in (x_0, x_0 + \eta)$, we have $|\Phi_{\theta_n}(x) - f(x)| \geq |y_n - b| - 2\delta$

Since μ has a continuous, strictly positive density at x_0, there exists $c > 0$ such that $\mu([x_0 - \eta, x_0]) \geq c\eta$ and $\mu([x_0, x_0 + \eta]) \geq c\eta$. Hence:

$$\mathcal{L}(\theta_n, \mathcal{T}) \geq c\eta \Big[(|y_n - a| - 2\delta)_+^2 + (|y_n - b| - 2\delta)_+^2 \Big].$$

Since $\{\Phi_{\theta_n}\}$ minimizes $\mathcal{L}$, and the loss contribution away from x_0 vanishes, the remaining loss near x_0 forces y_n to approximately minimize $|y - a|^2 + |y - b|^2$, which is a strictly convex quadratic with unique minimizer $y^* = \frac{a+b}{2}$. Since $\delta > 0$ was arbitrary (with $\eta = \eta(\delta)$ from equicontinuity), any accumulation point of $\{y_n\}$ must equal $\frac{a+b}{2}$. Therefore, $\Phi_{\theta_n}(x_0) \to$

$\frac{f(x_0^-)+f(x_0^+)}{2}$. □

(ii). Proposition 2 In addition to the state variables determining the equilibrium, a fake transient time dimension t is added. The system is solved as if there was a finite horizon T. This requires modifying the HJB equations by adding a time derivative when computing the drifts using Ito's lemma:

$$\mu^J J = \frac{\partial J}{\partial t} + \sum_i \mu^{x_i} \frac{\partial J}{\partial x_i} + \sum_{i,j} \frac{\sigma^{x_i}\sigma^{x_j}}{2} \frac{\partial^2 J}{\partial x_i \partial x_j}. \tag{E.1}$$

Proof. By rearranging (E.1) and replacing J with a general Φ_θ approximator, we can rewrite the HJB equations involving time derivatives as

$$\dot{\Phi}_\theta(x,t) := \partial_t \Phi_\theta(x,t) = \mathcal{F}[x,t,\Phi_\theta(x,t)] = \mu^{\Phi_\theta}\Phi_\theta - \sum_i \mu^{x_i} \frac{\partial \Phi_\theta}{\partial x_i} - \sum_{i,j} \frac{\sigma^{x_i}\sigma^{x_j}}{2} \frac{\partial^2 \Phi_\theta}{\partial x_i \partial x_j}$$

Step 1: The outer-loop operator. At each outer iteration τ, Algorithm 1 solves the evolution equation with terminal condition $\Phi_\tau(\cdot, T) = g_\tau$ (via the inner loop) and then sets $g_{\tau+1} = \Phi_\tau(\cdot, 0)$. This defines the iteration $g_{\tau+1} = S(g_\tau)$, where $S : C(\Omega) \to C(\Omega)$ is the terminal-to-initial solution map: $S(g)(x) := \Phi_g(x, 0)$, with $\Phi_g(\cdot, T) = g$.

Step 2: S is a contraction on a finite horizon. Consider the canonical structure $\partial_t \Phi = r\Phi - \mathcal{A}[\Phi]$, where $r > 0$ is the discount rate and $\mathcal{A}$ is the spatial differential operator. Let Φ_1, Φ_2 be two solutions with terminal condition g_1, g_2 respectively, and set $\delta(x,t) = \Phi_1(x,t) - \Phi_2(x,t)$. Then

$$\partial_t \delta = r\delta - \mathcal{A}[\delta]$$

Define $\tilde{\delta}(x,t) = e^{-rt}\delta(x,t)$, then: $\partial_t\tilde{\delta} = -\mathcal{A}[\tilde{\delta}]$, which, when read backward in time $s = T-t$, is a standard forward parabolic equation:

$$\partial_s\tilde{\delta} = \sum_i \mu^{x_i}\frac{\partial\tilde{\delta}}{\partial x_i} + \sum_{i,j}\frac{\sigma^{x_i}\sigma^{x_j}}{2}\frac{\partial^2\tilde{\delta}}{\partial x_i\partial x_j}.$$

By the maximum principle for parabolic equations, the supremum of $\left\|\tilde{\delta}\right\|$ is attained at the initial time $s = 0$ ($t = T$): $\left\|\tilde{\delta}(x,0)\right\|_\infty \leq \left\|\tilde{\delta}(x,T)\right\|_\infty$. Substituting back $\delta = e^{rt}\tilde{\delta}$:

$$\|S(g_1) - S(g_2)\|_\infty = \|\delta(x,0)\|_\infty \leq e^{-rT}\|g_1 - g_2\|_\infty.$$

Since $r > 0$ and $T > 0$, we have $\rho^S = e^{-rT} < 1$, so S is a contraction.

Step 3: Outer-loop convergence. Since $(C(\Omega), \|\cdot\|_\infty)$ is a complete metric space and S is contraction with $\rho^S < 1$, the Banach fixed-point theorem implies that the sequence $\{g_\tau\}$ defined by $g_{\tau+1} = S(g_\tau)$ converges uniformly to a unique fixed point g^* satisfying $S(g^*) = g^*$. This proves 1.

Step 4: Time-independence of the limit. Let Φ^* be the solution of the evolution equation with $\Phi^*(\cdot,T) = g^*$. The fixed-point condition $S(g^*) = g^*$ gives $\Phi^*(x,0) = g^*(x) = \Phi^*(x,T)$ for all $x \in \Omega$, so

$$\int_0^T \partial_t\Phi^*(x,t)\,dt = \Phi^*(x,T) - \Phi^*(x,0) = 0.$$

This proves 2. Because Φ^* satisfies $\partial_t\Phi^* = \mathcal{F}[\Phi^*]$ and the stationary equation $\mathcal{F}[x,\Phi] = 0$ has a unique solution Φ_s, which is trivially a fixed point of S, since a time-independent function has $S(\Phi_s) = \Phi_s$, uniqueness of the Banach fixed point implies $g^* = \Phi_s$. Therefore

$\Phi^* \equiv \Phi_s$ is independent of t and $\partial_t \Phi^* \equiv 0$.

Step 5: Vanishing time derivative. Uniform convergence $g_\tau \to g^*$, continuity of S, and the inner-loop accuracy condition (2) together yield $\lim_{\tau \to \infty} \partial_t \Phi_\tau = \partial_t \Phi^* = 0$. □

F Software and replication package

The python library along with implementation details can be found in `https://github.com/rotmanfinhub/deep-macrofin`. We provide a sample Jupyter notebook that demonstrates the parsing of equations at `https://github.com/rotmanfinhub/deep-macrofin/blob/main/examples/time_step/ditella.ipynb`. A comprehensive documentation of the library is available at `https://rotmanfinhub.github.io/deep-macrofin/`. Additionally, the replication package for the experiments conducted in this paper is available at `https://github.com/yuntaowu2000/active_learning_paper`.

References

Azinovic, Markon and Jan Zemlicka, "Intergenerational consequences of rare disasters," 2023.

Azinovic, Marlon, Luca Gaegauf, and Simon Scheidegger, "Deep equilibrium nets," *International Economic Review*, 2022, *63* (4), 1471–1525.

Bischof, Rafael and Michael A. Kraus, "Multi-Objective Loss Balancing for Physics-Informed Deep Learning," *Computer Methods in Applied Mechanics and Engineering*, 2025, *439*, 117914.

Bretscher, Lorenzo, Jesús Fernández-Villaverde, and Simon Scheidegger, "Ricardian Business Cycles," *SSRN Electronic Journal*, 11 2022.

Brumm, Johannes and Simon Scheidegger, "Using Adaptive Sparse Grids to Solve High-Dimensional Dynamic Models," *Econometrica*, 2017.

Brunnermeier, Markus K. and Yuliy Sannikov, "A Macroeconomic Model with a Financial Sector," *American Economic Review*, 2 2014, *104* (2), 379–421.

Brunnermeier, Markus K and Yuliy Sannikov, "Macro, Money and Finance: A Continuous Time Approach," Working Paper 22343, National Bureau of Economic Research 6 2016.

Bungartz, Hans Joachim, Alexander Heinecke, Dirk Pflüger, and Stefanie Schraufstetter, "Option pricing with a direct adaptive sparse grid approach," in "Journal of Computational and Applied Mathematics" 2012.

Cipolla, Roberto, Yarin Gal, and Alex Kendall, "Multi-task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in "2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition" 2018, pp. 7482–7491.

D'avernas, Adrien, Damon Petersen, and Quentin Vandeweyer, "A Solution Method for Continuous-Time Models," 2023.

Duarte, Victor, Diogo Duarte, and Dejanir H Silva, "Machine Learning for Continuous-Time Finance," 2024.

Décamps, Jean-Paul, Sebastian Gryglewicz, Erwan Morellec, and Stéphane Villeneuve, "Corporate Policies with Permanent and Transitory Shocks," *The Review of Financial Studies*, 2017, *30* (1), 162–210.

Fernández-Villaverde, Jesús, Samuel Hurtado, and Galo Nuño, "Financial Frictions and the Wealth Distribution," *Econometrica*, 2023, *91*, 869–901.

Gopalakrishna, Goutham, "A Macro-Finance model with Realistic Crisis Dynamics," in "Proceedings of Paris December 2021 Finance Meeting EUROFIDAI - ESSEC" 11 2020. Swiss Finance Institute Research Paper No. 20-96.

_ , Zhouzhou Gu, and Jonathan Payne, "Institutional Asset Pricing with Segmentation and Household Heterogeneity.," *Princeton Univeristy Working Paper*, 2026.

Gu, Zhouzhou, Mathieu Laurière, Sebastian Merkel, and Jonathan Payne, "Global Solutions to Master Equations for Continuous Time Heterogeneous Agent Macroeconomic Models," 6 2024.

Han, Jiequn, Arnulf Jentzen, and Weinan E, "Solving high-dimensional partial differential equations using deep learning," *Proceedings of the National Academy of Sciences*, 2018, *115* (34), 8505–8510.

_ , Yucheng Yang, and Weinan E, "DeepHAM: A Global Solution Method for Heterogeneous Agent Models with Aggregate Shocks," *SSRN Electronic Journal*, 12 2021.

Hornik, Kurt, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, 1991, *4* (2), 251–257.

_ , Maxwell Stinchcombe, and Halbert White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, 1989, *2* (5), 359–366.

Huang, Ji, "A Probabilistic Solution to High-Dimensional Continuous-Time Macro and Finance Models," 2023.

Lu, Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis, "DeepXDE: A Deep Learning Library for Solving Differential Equations," *SIAM Review*, 2021, *63* (1), 208–228.

Maliar, Lilia and Serguei Maliar, "Merging simulation and projection approaches to solve high-dimensional problems with an application to a new Keynesian model," *Quantitative Economics*, 3 2015, *6*, 1–47.

_ , _ , and Pablo Winant, "Deep learning for solving dynamic economic models.," *Journal of Monetary Economics*, 2021, *122*, 76–101.

Martin, Ian, "The Lucas Orchard," *Econometrica*, 1 2013, *81*, 55–111.

Merkel, Sebastian, "The Macro Implications of Narrow Banking: Financial Stability versus Growth," 2020.

Payne, Jonathan, Adam Rebei, and Yucheng Yang, "Deep Learning for Search and Matching Models," *SSRN Electronic Journal*, 2 2024.

Petersen, Philipp and Jakob Zech, "Mathematical theory of deep learning," 2024.

Raissi, M, P Perdikaris, and G E Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 2019.

Sauzet, Maxime, "Projection Methods via Neural Networks for Continuous-Time Models," 2021.

Schaab, Andreas and Allen Zhang, "Dynamic Programming in Continuous Time with Adaptive Sparse Grids," *SSRN Electronic Journal*, 5 2022.

Sirignano, Justin and Konstantinos Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, 2018.

Tella, Sebastian Di, "Uncertainty Shocks and Balance Sheet Recessions," *Journal of Political Economy*, 2017, *125* (6), 2038–2081.

ALIENs for Continuous Time Economies

Internet Appendix

This online appendix provides the optimization problems for each of the economic models, while we refer the readers to the original papers for the complete setup.

A Free boundary models

The benchmark free boundary model follows Brunnermeier and Sannikov (2014). There are two types of risk-neutral agents, experts and households, in the economy. Both types of agents $j \in \{e, h\}$ can hold capital $k_{j,t}$ at time t, and produce output at rate

$$y_{j,t} = a_j k_{j,t}.$$

Experts are more productive, *i.e.* $a_e > a_h$. They also hold a risk-free asset.

Capital evolves according to a geometric Brownian motion:

$$\frac{dk_{j,t}}{k_{j,t}} = (\Phi(\iota_{j,t}) - \delta_j)dt + \sigma dZ_t^k,$$

where $\iota_{j,t}$ is the investment rate per unit of capital, δ_j is depreciation rate ($\delta_h \geq \delta_e$), Φ represents an investment functional with adjustment costs, and dZ_t^k is an exogenous aggregate Brownian shock.

Experts and households have discount rates ρ_j, with $\rho_e > \rho_h = r$, where r denotes the

risk-free rate. Let $c_{j,t}$ denote cumulative consumption, $n_{j,t}$ net worth, and $x_{j,t}$ the fraction of wealth invested in capital by agent j at time t, Experts and households earn returns $r^k_{e,t}$ and $r^k_{h,t}$ from capital respectively, depending on the equilibrium market price of capital q_t at time t. Each agent maximizes expected lifetime utility subject to their respective budget constraints:

$$\begin{aligned} &\max_{x_{j,t}\geq 0, dc_{j,t}\geq 0, \iota_{j,t}} E\left[\int_0^\infty e^{-\rho_j t} dc_{j,t}\right] \\ \text{s.t.} \quad &\frac{dn_{j,t}}{n_{j,t}} = x_{j,t} dr^k_{j,t} + (1 - x_{j,t}) r dt - \frac{dc_{j,t}}{n_{j,t}}, \\ &n_{j,t} \geq 0 \end{aligned}$$

In an equilibrium,

1. Initial net worths satisfy $n_{j,0} = k_{j,0} q_0$ for each individual agent.

2. Each expert and each household solve their optimization problems given prices.

3. Consumption and capital markets clear.

With some algebra, one can deduce the one-dimension model with the system of ordinary differential equation presented.

The model is extended to two-dimension by varying the expert's productivity $a_{e,t}$, following Gopalakrishna (2020). Now $a_{e,t}$ follows an Ornstein-Uhlenbeck process with stochastic volatility:

$$da_{e,t} = \pi(\hat{a}_e - a_{e,t})dt + \nu \sigma_{ae,t} dZ^a_t,$$

where dZ_t^a has a correlation ϕdt with dZ_t^k, π is a persistence parameter, and $\hat{a}_e$ is the mean value.

The agents have recursive utility:

$$U_{j,t} = E_t\left[\int_t^\infty f(c_{j,s}, U_{j,s})ds\right]$$
$$\text{with } f(c_{j,t}, U_{j,t}) = (1-\gamma)\rho U_{j,t}\left(\log(c_{j,t}) - \frac{1}{1-\gamma}\log\left((1-\gamma)U_{j,t}\right)\right),$$

where γ and ρ are risk aversion and the discount rate coefficients respectively.

At each time instance dt, agents are born and die with a probability λ_d, and a fraction $\tau_t dt$ of experts become households. Let τ' denote the exit time, exponentially distributed with rate τ_t. The experts maximizes their utilities, subject to wealth constraints, starting at initial wealth $w_{e,0}$:

$$\sup_{c_{e,t},k_{e,t},\chi_t} E_t\left[\int_t^{\tau'} f(c_{e,s}, U_{e,s})ds + U_{h,\tau'}\right]$$
$$\text{s.t.} \quad \frac{dw_{e,t}}{w_{e,t}} = \left(r_t - \frac{c_{e,t}}{w_{e,t}} + \frac{q_t k_{e,t}}{w_{e,t}}\left(\mu_{e,t}^R - r_t - (1-\chi_t)\epsilon_{h,t}\right)\right)dt + \sigma_{w_e,t}\left((\sigma + \sigma_t^{q,k})dZ_t^k + \sigma_t^{q,a}dZ_t^a\right),$$

where $\frac{q_t k_{e,t}}{w_{e,t}}$ and χ_t denote the fraction of wealth invested in capital and inside equity share. The experts earn an expected excess return of $\mu_{e,t}^R - r_t$ from risky assets, but they have to pay equity investors $(1-\chi_t)\epsilon_{h,t}$ with premium $\epsilon_{h,t}$. Households solve a similar optimization problem without equity issuance:

$$\sup_{c_{h,t},k_{h,t}} E_t\left[\int_t^\infty f(c_{h,s}, U_{h,s})ds\right]$$

$$\text{s.t.} \quad \frac{dw_{h,t}}{w_{h,t}} = \left(r_t - \frac{c_{h,t}}{w_{h,t}} + \frac{q_t k_{h,t}}{w_{h,t}} (\mu^R_{h,t} - r_t) \right) dt + \sigma_{w_h,t} \left((\sigma + \sigma^{q,k}_t) dZ^k_t + \sigma^{q,a}_t dZ^a_t \right)$$

$$k_{h,t} \geq 0 \text{ (no-shorting constraint)}$$

In the equilibrium, capital and goods markets clear, and the total wealth is invested in the capital.

B Optimal stopping problem

This model is based on Décamps et al. (2017). The probability space is $(\Omega, \mathcal{F}, \mathbb{F}, P)$ with filtration $\mathbb{F} = \{\mathcal{F}_t : t \geq 0\}$. The agents are risk neutral and discount rate r is constant. A firm owns an option to invest in a risky project. Let A_t denote the productivity of assets. It is affected by permanent shocks and governed by a geometric Brownian motion:

$$dA_t = \mu A_t dt + \sigma_A A_t dW_t^P$$

The dynamics of cash flows can be rewritten as

$$dX_t = \alpha A_t dt + \sigma_X A_t \left(\rho dW_t^P + \sqrt{1-\rho^2} dW_t^T \right),$$

where W^T is independent from W^P.

Let M_t denote the firm's cash holdings, L_t, E_t, and Φ_t denote nondecreasing processes that represent the cumulative dividend paid to shareholders, the cumulative gross external financing raised from outside investors, and the cumulative fixed cost of financing. The firm chooses the optimal dividend (L), financing from investors (e_n), and default policies (τ_n) that maximize shareholder's value function:

$$V(a,m) = \sup_{L,(\tau_n)_{n\geq 0},(e_n)_{n\geq 1}} E_{a,m}\left[\int_0^{\tau_0} e^{-rt}(dL_t - dE_t) + e^{-r\tau_0}\left(\frac{\omega \alpha A_{\tau_0}}{r-\mu} + M_{\tau_0} \right) \right]$$

With value matching and change of variable $c = \frac{m}{a}$ that represents the scaled cash holdings of the firm and $F(c)$ that is the scaled value function, we reach the benchmark modified

optimization problem:

$$(r-\mu)F(c) = (\alpha + c(r-\lambda-\mu))F'(c) + \frac{1}{2}(\sigma_A^2 c^2 - 2\rho\sigma_A\sigma_X c + \sigma_X^2)F''(c), c < c^*$$

$$F(c) = F(c^*) + c - c^*, c \geq c^*$$

$$F(0) = \max\left\{\max_{c\in[0,\infty]}(F(c) - p(c+\phi)), \frac{\omega\alpha}{r-\mu}\right\}$$

$$F'(c^*) = 1 \quad F''(c^*) = 0$$

The goal is to find the optimal cash holding c^* and solve for $F(c)$.

C Stochastic volatility model

There are two types of agents: experts (intermediary) who can trade and use capital to produce and households that finance them. Capital is exposed to both aggregate and (expert-specific) idiosyncratic Brownian TFP shocks. The change in expert's effective capital in a short period of time is

$$\frac{dk_t}{k_t} = g_t dt + \sigma dZ_t + v_t dW_t,$$

where g_t is the growth rate for the capital stock, $\iota(g)$ is the investment function, Z is an aggregate Brownian motion, and W is an idiosyncratic Brownian motion for expert in a probability space $(\Omega, \mathcal{F}_t, \mathcal{P})$. The exposure of capital to aggregate risk $\sigma > 0$ is constant, but the exposure to idiosyncratic risk v_t follows an exogenous stochastic process,

$$dv_t = \lambda(\bar{v} - v_t)dt + \bar{\sigma_v}\sqrt{v_t}dZ_t,$$

where $\bar{v}$ is the long-run mean and λ is the mean reversion parameter. The loading of the idiosyncratic volatility of capital on aggregate risk is $\bar{\sigma_v} < 0$, so Z is an aggregate shock that increases the effective capital stock and reduces idiosyncratic risk. This v_t extends the dimensionality of the economic model to two.

Both experts and households have Epstein-Zin preferences with the same discount rate ρ, risk aversion γ, and elasticity of intertemporal substitution (EIS) ψ^{-1}. The utility function is given as

$$U_t^j = \mathbb{E}_t \left[\int_t^\infty f(c_s, U_s) ds \right],$$

where the normalized aggregator of consumption and continuation value is

$$f(c,U)=\frac{1}{1-\psi}\left(\frac{\rho c^{1-\psi}}{[(1-\gamma)U]^{(\gamma-\psi)/(1-\gamma)}}-\rho(1-\gamma)U\right).$$

Experts trade capital continuously at price p, whose process can be conjectured as:

$$\frac{dp_t}{p_t}=\mu_{p,t}dt+\sigma_{p,t}dZ_t.$$

The total value of the aggregate capital stock is $p_t k_t$, and it constitutes the total wealth of the economy. The financial market has stochastic discount factor η with:

$$\frac{d\eta_t}{\eta_t}=-r_t dt-\pi_t dZ_t,$$

where r_t is the risk-free interest rate and π_t is the price of aggregate risk Z.

The cumulative return from investing a dollar in capital for expert is R_t^k, with

$$dR_t^k=\left(\frac{a-\iota(g_t)}{p_t}+g_t+\mu_{p,t}+\sigma\sigma_{p,t}\right)dt+(\sigma+\sigma_{p,t})dZ_t+v_t dW_t.$$

Experts solve the following optimization problem:

$$\begin{aligned}&\max_{e,g,k,\theta} U(e)\\ \text{s.t. }\frac{dn_t}{n_t}=&(\mu_{n,t}-\hat{e}_t)dt+\sigma_{n,t}dZ_t+\tilde{\sigma}_{n,t}dW_t,\end{aligned}$$

where n is the expert's net worth, $\sigma_{n,t}$ is their exposure to aggregate risk, and $\tilde{\sigma}_{n,t}$ is their

exposure to idiosyncratic risk. $\tilde{\sigma}_{n,t}$ comes from the fraction $\phi \in (0,1)$ (moral hazard) of their return that they keeps. They sells the rest of $1-\phi$ on the market. θ is expert's position in the normalized market index.

The value function for an expert with net worth n is

$$V(n) = \frac{(\xi_t n)^{1-\gamma}}{1-\gamma},$$

for some stochastic process ξ_t such that

$$\frac{d\xi_t}{\xi_t} = \mu_{\xi,t}dt + \sigma_{\xi,t}dZ_t.$$

The experts retire with independent Poisson arrival rate $\tau > 0$ and become households. The turnover gives a modified HJB equation for experts:

$$\frac{\rho}{1-\psi} = \max\left\{\frac{\hat{e}^{1-\psi}}{1-\psi}\rho\xi^{\psi-1} + \frac{\tau}{1-\gamma}\left(\left(\frac{\zeta}{\xi}\right)^{1-\gamma} - 1\right) + \mu_n - \hat{e} + \mu_\xi - \frac{\gamma}{2}\left(\sigma_n^2 + \sigma_\xi^2 - 2\frac{1-\gamma}{\gamma}\sigma_n\sigma_\xi + \tilde{\sigma_n}^2\right)\right\}.$$

Households solve the following optimization problem:

$$\max_{c,\sigma_w} U(c)$$
$$\text{s.t. } \frac{dw_t}{w_t} = (r_t + \sigma_{w,t}\pi_t - \hat{c}_t)dt + \sigma_{w,t}dZ_t,$$

where w is the wealth of households, and $\sigma_{w,t}$ is their exposure to aggregate risk.

The value function for a household with net wealth w is

$$V(w) = \frac{(\zeta_t w)^{1-\gamma}}{1-\gamma},$$

for some stochastic process ζ_t such that

$$\frac{d\zeta_t}{\zeta_t} = \mu_{\zeta,t}dt + \sigma_{\zeta,t}dZ_t.$$

The HJB equation associated with households' problem is

$$\frac{\rho}{1-\psi} = \max\left\{\frac{\hat{c}^{1-\psi}}{1-\psi}\rho\zeta^{\psi-1} + \mu_w - \hat{c} + \mu_\zeta - \frac{\gamma}{2}\left(\sigma_w^2 + \sigma_\zeta^2 - 2\frac{1-\gamma}{\gamma}\sigma_w\sigma_\zeta\right)\right\}.$$

The state space is defined by two state variables: $x = n/pk$, representing the net worth of experts relative to the total value of assets in equilibrium; and v, indicating the experts' exposure to idiosyncratic risk. $(x, v) \in (0, 1) \times (0, \infty)$.

Market clearing condition at equilibrium gives the following conditions:

$$a - \iota = p(\hat{e}x + \hat{c}(1-x))$$

$$\sigma + \sigma_p = \sigma_n x + \sigma_w(1-x)$$

$$\frac{a-\iota}{p} + g + \mu_p + \sigma\sigma_p - r = (\sigma + \sigma_p)\pi + \gamma\frac{1}{x}(\phi v)^2$$

D N-tree

This model is based on Martin (2013). There are N assets with positive dividends y_{jt} for $j = 1, 2, ..., N$. The dividends y_j follow a geometric Brownian motion:

$$\frac{dy_j}{y_j} = \mu_j dt + \sigma_j dZ_j,$$

with constant μ_j and σ_j for each asset j. In addition, there is a risk-free asset with rate r_t. The markets are complete and there is a unique SDF whose dynamics is

$$\frac{d\xi_t}{\xi_t} = -r_t dt - \sum_j \zeta_{jt} dZ_j,$$

where ζ_j is the state price for shock Z_j.

There is a representative agent with consumption c_t (sum of the dividends), with relative risk aversion γ and discount rate ρ. The agent maximizes their CRRA utility:

$$U_t = E_t \left[\int_0^\infty e^{-\rho t} \frac{c_t^{1-\gamma}}{1-\gamma} dt \right],$$

subject to the budget constraint:

$$\frac{dn_t}{n_t} = \left(r_t - \frac{c_t}{n_t} + \sum_j w_{jt}(r_{jt} - r_t) \right) dt + \sum_j w_{jt} \sum_k \sigma^{q_{jk}} dZ_k,$$

where q_{jt} is the price of asset j with predetermined dynamics, r_{jt} is the return on asset j, and w_{jt} is the weight on asset j. By defining the state variable to be the share of consumption

$z_j = \frac{y_j}{\sum_j y_j}$, we reach the system of equations provided in the paper appendix.